

Table Of Contents

General Introduction	1
1 General Context	3
Introduction	3
1.1 Company Presentation	3
1.2 Project Presentation	4
1.2.1 Study Of The Existing System	4
1.2.2 Critics Of The Existing System	5
1.2.3 Proposed Solution	5
1.3 Work Methodology	6
1.3.1 Agile Process	6
1.3.2 The Adopted Method : Scrum	7
Conclusion	7
2 DevOps Fundamentals and Tooling	8
Introduction	8
2.1 DevOps Overview	8
2.1.1 Definition of DevOps	8
2.1.2 DevOps Toolchain	9
2.2 DevOps Concepts and Tool Comparisons	9
2.2.1 Continuous Integration / Continuous Deployment (CI/CD)	9
2.2.2 Virtualization and Containerization	10
2.2.3 Container Orchestration	12
2.2.4 GitOps and Deployment Automation	13
2.2.5 Cloud	13

2.2.6	Infrastructure as Code (IaC)	14
2.2.7	Monitoring and Observability	15
2.3	Selected Tools Overview	16
2.3.1	Docker	16
2.3.2	Jenkins	17
2.3.3	Kubernetes	17
2.3.4	ArgoCD	20
2.3.5	Amazon Web Services (AWS)	21
2.3.6	Terraform	22
2.3.7	Prometheus	23
2.3.8	Grafana	24
	Conclusion	24
3	Planning	25
	Introduction	25
3.1	Requirement Analysis	25
3.1.1	Identifying Actors	25
3.1.2	Functional Requirements	25
3.1.3	Non-Functional Requirements	26
3.2	Requirement Specification	26
3.2.1	General Definitions and Tools	26
3.2.2	Global Use Case Diagram	27
3.3	Project Planning with Scrum	28
3.3.1	Product Backlog	28
3.3.2	Sprints Planning	31
3.4	Proposed Architecture	32
3.4.1	AWS Architecture	32
3.4.2	DevOps Architecture	33
3.4.3	Kubernetes Architecture	34
	Conclusion	35

4 Sprint 1: Settings Up Infrastructure as Code (IaC) and Automation	
Tools	36
Introduction	36
4.1 Sprint 1 Backlog Product	36
4.2 Sprint 1 Realisation	37
Conclusion	49
5 Sprint 2: Implementing CI pipeline and Docker Integration	50
Introduction	50
5.1 Sprint 2 Backlog Product	50
5.2 Sprint 2 Realisation	51
Conclusion	62
6 Sprint 3: Designing a Highly Available and Auto-Scaling Kubernetes	
Cluster	63
Introduction	63
6.1 Sprint 3 Backlog Product	63
6.2 Sprint 3 Realisation	64
Conclusion	75
7 Sprint 4: From Deployment to Insight: GitOps with ArgoCD and	
Real-Time Observability	76
Introduction	76
7.1 Final Sprint Product Backlog	76
7.2 Final Sprint Realisation	77
Conclusion	85
General Conclusion and Perspectives	86

List of Figures

1.1	JobGate Logo	4
1.2	Current Project	4
1.3	Agile Overview	6
1.4	Scrum Process	7
2.1	DevOps Workflow	9
2.2	Bare Metal vs Virtualization	11
2.3	Containerization	11
2.4	Hybred Solution	12
2.5	Kubernetes Architecture	20
2.6	GitOps deployment approach with ArgoCD	20
2.7	Terraform workflow	23
2.8	Prometheus Architecture	24
3.1	use case diagram	27
3.2	AWS Architecture	32
3.3	DevOps Architecture	33
3.4	K8s Architecture	34
4.1	VPC	37
4.2	Ubuntu AMI	38
4.3	Jenkins EC2 Instance	39
4.4	SonarQube EC2 Instance	39
4.5	Nexus EC2 Instance	40
4.6	Unlock Jenkins	40
4.7	Plugins Suggestion	41

4.8 Create Admin User	41
4.9 Jenkins Dashboard	42
4.10 Required Plugins for CI pipeline	42
4.11 jdk 11	43
4.12 maven 3	43
4.13 Node 12	44
4.14 Sonar-Scanner	44
4.15 Github Credentials	45
4.16 Accessing SonarQube	45
4.17 Sonar Token	46
4.18 Sonar Credentials	46
4.19 Configure SonarQube server in Jenkins	47
4.20 Creating Repository for SCRUM-UI	47
4.21 Adding Npm Bearer Token Realm	48
4.22 SonarQube and JaCoCo plugins	48
4.23 Nexus Plugins	49
4.24 Nexus distribution Management	49
4.25 Nexus URL in frontend	49
5.1 Creating Pipeline Type named pfe-2025	51
5.2 Max of builds to keep	52
5.3 Pipeline Script From SCM	52
5.4 Jenkinsfile Part 1	53
5.5 Jenkinsfile Part 2	53
5.6 Jenkinsfile Part 3	54
5.7 Jenkinsfile Part 4	54
5.8 CI pipeline Result	55
5.9 SonarQube Results	55
5.10 SCRUM-UI Artifact	55
5.11 SCRUM-APP Artifact	56
5.12 Scrum-ui Dockerfile	56

5.13 Scrum-app Dockerfile	57
5.14 Docker plugin	57
5.15 Docker Tool Configuration	58
5.16 Docker Tool Configuration	58
5.17 Scrum-ui dockerhub repository	59
5.18 Scrum-app dockerhub repository	59
5.19 used environment variables for docker integration	60
5.20	60
5.21 Docker Login Stage	61
5.22 Docker Push Images Stage	61
5.23 Scrum-ui docker image	61
5.24 Scrum-app docker image	62
5.25 Improved CI pipeline result	62
6.1 Updating VPC and Subnets	64
6.2 IAM for ALB Controller and Cluster Autoscaler	65
6.3 Bastion EC2 instance	66
6.4 Control Plane Provisionning	66
6.5 Aws Launch Template part 1	67
6.6 Aws Launch Template part 2	68
6.7 Worker Nodes Auto Scaling Group	69
6.8 patch worker nodes script	70
6.9 ingress yaml file	71
6.10 scrum-app service yaml file	71
6.11 scrum-app deployment yaml file	72
6.12 scrum-app hpa yaml file	72
6.13 scrum-postgres deployment yaml file	73
6.14 scrum-postgres service yaml file	73
6.15 scrum-postgres vpa yaml file	74
6.16 scrum-ui deployment yaml file	74
6.17 scrum-ui hpa yaml file	75

6.18 scrum-ui service yaml file	75
7.1 GitOps Repository Structure	77
7.2 Updating Docker Version in GitOps Repo	78
7.3 Email Notification	78
7.4 Setting GitOps Repository	79
7.5 GitOps Repository Successfully Setup	79
7.6 ArgoCD Application Created from Private GitOps Repository	79
7.7 K8s components displayed in ArgoCD UI	80
7.8 Initial Nodes in AWS Console	80
7.9 Current Kubernetes Cluster Nodes	81
7.10 Viewing Current Pods	81
7.11 Initial HPA Configuration for Scrum UI and Scrum App	81
7.12 Verification of AWS Load Balancer Controller	81
7.13 Verification of Metric Server, CA and VPA	81
7.14 Stress Test on Pod to Trigger Autoscaling	82
7.15 Pending State Pods	82
7.16 Increased Worker Nodes	82
7.17 Nodes Terminal Verification	83
7.18 New Pods Created Automatically	83
7.19 Back With Initial Nodes State	83
7.20 Scaling Down Nodes	83
7.21 Cluster Overview Dashboard	84
7.22 Worker Node CPU Usage	84

List of Tables

1.1	JobGate Overview	3
2.1	CI Tools Comparison	10
2.2	Comparison of Virtualization and Containerization	12
3.1	Full Product Backlog	28
3.2	Summary of the Four Sprints	31
4.1	Sprint 1 Backlog Product	36
5.1	Sprint 2 Backlog Product	50
6.1	Sprint 3 Backlog Product	63
7.1	Sprint 4 Product Backlog	76

List of Acronyms

ALB	Application Load Balancer
ASG	Auto Scaling Group
AWS	Amazon Web Services
CA	Cluster Autoscaler
CD	Continuous Deployment
CI	Continuous Integration
CPU	Central Processing Unit
DevOps	Development and Operations
EC2	Elastic Compute Cloud
HPA	Horizontal Pod Autoscaler
HTTP	HyperText Transfer Protocol
IaC	Infrastructure as Code
IAM	Identity and Access Management
IG	Internet Gateway
K8s	Kubernetes
NAT	Network Address Translation
PromQL	Prometheus Query Language
TSDB	Time Series Database
VPA	Vertical Pod Autoscaler
VPC	Virtual Private Cloud
VM	Virtual Machine
YAML	Yet Another Markup Language

General Introduction

In recent years, the world of software development and IT operations has changed significantly. In the past, teams worked in silos, developers focused only on writing code, while operations teams handled deployment and infrastructure. This separation created many challenges: slow releases, communication gaps, frequent bugs, and complicated manual processes that were hard to scale or maintain.

The rise of cloud computing and DevOps practices has transformed the way modern applications are built and deployed. With the cloud, companies can access flexible, on-demand infrastructure and pay only for what they use. At the same time, DevOps has brought development and operations teams closer together, promoting automation, collaboration, and continuous delivery.

In this new approach, the goal is to deliver software faster and with better quality. DevOps encourages automation across the entire the software lifecycle starting from writing code, testing it, building it into containers, and finally deploying it to production with real-time monitoring and feedback. It also relies on key practices such as Infrastructure as Code (IaC), Continuous Integration/Continuous Deployment (CI/CD), containerization, orchestration and GitOps to make systems scalable and reliable.

It is within this modern context that we carried out our final year project at JobGate.

Following the agile methodology, we organized our work into a product backlog and completed it through four sprints. This report is structured in seven chapters:

- **General Context :** Presents the organization, outlines the problematic, solution, selected methodology.
- **DevOps Fundamentals and Tooling :** Explains the core DevOps concepts, practices, and provides an overview of the tools and technologies adopted throughout the project.
- **Planning :** Describes the project management approach, including requirement analysis, requirement specification, backlog creation, sprint planning and the proposed architecture.
- **Sprint 1: Settings Up Infrastructure as Code (IaC) and Automation Tools:** focused on provisioning and setting up essential automation tools like Jenkins, SonarQube, Nexus.
- **Sprint 2: Implementing CI pipeline and Docker Integration :** covered the implementation of a CI pipeline, and the Docker integration.
- **Sprint 3: Designing a Highly Available and Auto-Scaling Kubernetes Cluster :** was dedicated to creating a kubernetes cluster across private subnets, ensuring high availability and auto-scaling when traffic increases.

- **Sprint 4: From Deployment to Insight: GitOps with ArgoCD and Real-Time Observability :** introduced GitOps using ArgoCD for automated application deployment and real-time observability using Prometheus and Grafana for monitoring .

Through this project, we had the opportunity to apply DevOps practices in a real-world environment. It helped us build a strong understanding of cloud infrastructure, automation, and modern software delivery workflows. This hands-on experience allowed us to become more confident as future DevOps engineers, capable of building scalable, secure, and reliable solutions using the best tools available today.

Chapter 1

General Context

Introduction

In this very first chapter we will start by providing details about the company structure including its business activities. The next step analyzes how the current system functions, followed by an identification of system problems before suggesting the proposed solution. Also we will define our work methodology for this project and finally a conclusion for quick recap.

1.1 Company Presentation

JobGate is a technology company founded in 2020, specializing in training and recruitment. Its mission is to empower young professionals with the knowledge, skills, and expertise needed to succeed in the job market.

Table 1.1: JobGate Overview

CORPORATE NAME	JobGate
LEGAL STATUS	SARL
ADRESSE	12 Abdelkadeur Daghrir Street, Hammam Sousse – Sousse 4011
PHONE	(+216) 29 781 479 / (+216) 36 096 039
E-MAIL	contact@jobgate.tn
SITE	www.jobgate.tn
ACTIVITIES	• Preparing IT professionals for the most in-demand roles in the job market. • Career transition support for those looking to switch fields and move into a new domain.



Figure 1.1: JobGate Logo

1.2 Project Presentation

The figure below shows a visual overview of an example of the project carried out by JobGate company.

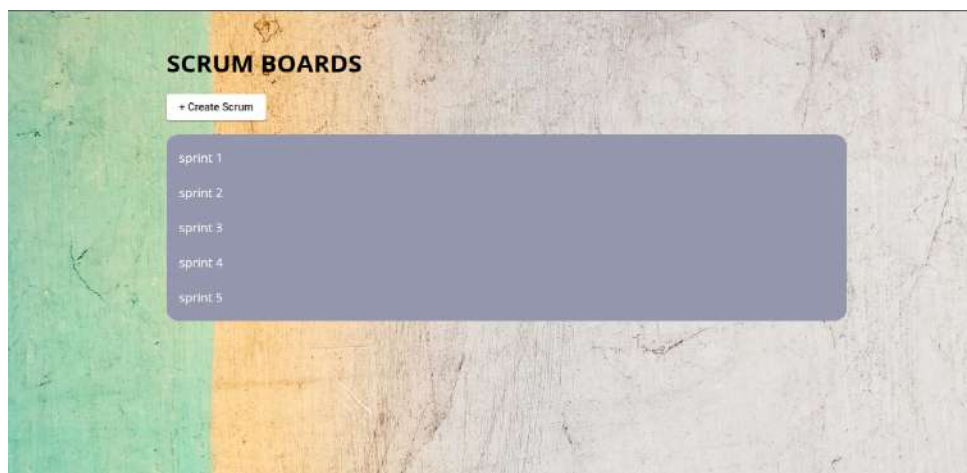


Figure 1.2: Current Project

1.2.1 Study Of The Existing System

The current system is mostly based on manual work. Developers write the code, test it, and deploy it by hand. This means every step takes time and can be different each time, depending on who does it. There are no automated pipelines to check the code, build it, or deploy it. This slows down the entire development process and increases the chances of making mistakes.

Also, the application is deployed directly onto servers without using containers. This makes it hard to ensure the app works the same way in all environments — for example, on the developer's machine, in testing, and in production. Sometimes the app works in one place but not in another, which creates confusion and wastes time.

The infrastructure setup is also done manually. Creating servers, installing dependencies, and setting configurations all depend on individual choices and memory, which can cause inconsistencies. This lack of standardization makes it difficult to reproduce the same environment every time, leading to unexpected bugs or delays when setting up new environments.

Lastly, the system doesn't include proper monitoring tools. When a problem happens, it's hard to know what went wrong or when it started. This makes it difficult to fix issues quickly or prevent them from happening again.

1.2.2 Critics Of The Existing System

Time, quality, and cost are key factors for a company's success. However, the current system faces many problems that make it less efficient and harder to manage.

One major issue is the lack of communication and collaboration between the development and operations teams. Developers focus on building new features and fixing bugs, while the operations team wants to keep the infrastructure stable. These teams often work separately and don't fully understand each other's challenges. This creates confusion and delays.

Another problem is that the build, test, and deployment processes are all done manually. This not only slows down the delivery of new updates, but also increases the chances of human error. Releasing updates on time becomes difficult, and quality may suffer.

Also, the application is deployed directly on servers without using containers. This causes the classic issue: "It works on my machine," meaning the app works in one environment but fails in another. The system lacks portability, making it harder to ensure consistent performance across environments.

The infrastructure is also managed manually. Each time a new server is needed, it has to be purchased, delivered to the data center, and configured. This process is slow, expensive, and hard to scale. When the business grows and traffic increases, the server may not handle the load, leading to poor performance. On the other hand, when traffic decreases, resources remain underused, wasting money and effort.

There is no automation or standard setup, so every team member might configure things differently. This makes the system harder to maintain and troubleshoot. Additionally, there are no proper monitoring tools in place, so problems are often discovered too late after users are already affected.

Overall, the current system is not flexible, scalable, or efficient enough to meet modern demands.

1.2.3 Proposed Solution

The proposed solution adopts a modern DevOps approach by integrating automation tools and cloud infrastructure to improve the reliability, speed, and manageability of the system. Here's the plan:

To begin with, Jenkins will be used for Continuous Integration (CI). It will automate the process of checking code quality, building the application, and running tests, ensuring that issues are caught early and development is streamlined.

For Continuous Deployment (CD), we will use ArgoCD. It will manage the deployment of applications into the Kubernetes cluster, keeping everything in sync with the Git repository and enabling GitOps-based workflows.

The application will be containerized using Docker, making it consistent across different environments and easy to ship and deploy.

It is then deployed to a Kubernetes cluster hosted on AWS, benefiting from cloud flexibility and scalability.

To handle varying levels of traffic, autoscaling has been implemented. This ensures the system automatically adjusts resources based on demand, maintaining performance while optimizing costs.

An Application Load Balancer (ALB) is used to manage and distribute incoming traffic efficiently, enhancing both availability and reliability of the application.

The entire infrastructure is provisioned using Terraform, allowing infrastructure to be defined and managed as code, improving repeatability and reducing manual errors.

For observability, Prometheus and Grafana are integrated to monitor system performance and visualize metrics, enabling proactive troubleshooting and insights.

1.3 Work Methodology

1.3.1 Agile Process

An agile method is an iterative and incremental approach, which is carried out in a collaborative spirit. Iterations in the agile environment are called sprints and last from one to four weeks. Their goal is to release a new software feature. Each of these iterations includes planning, requirements, design, code, testing, and documentation.

The team working on an agile project should consist at least of the developers who create it and the people who define the direction and goals of the project, such as the project manager and customers.

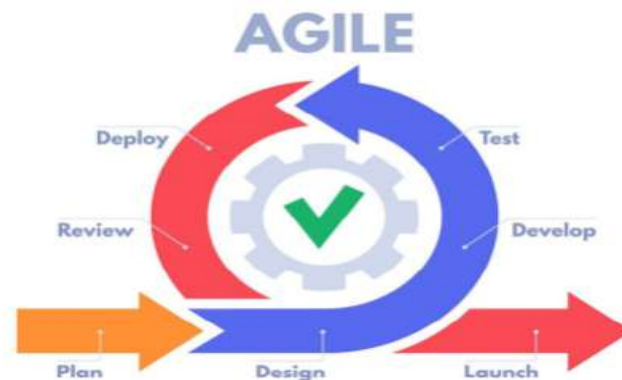


Figure 1.3: Agile Overview

- **Stakeholder Engagement and Satisfaction:**

Agile development process encourages the stakeholder’s engagement through regular meetings. This active involvement permits the developers to fully understand the project’s requirements, therefore ensuring the client’s satisfaction.

- **Transparency:**

The agile method focuses on involving the client through each and every step of the development process. This involvement gives the client full visibility over the product from early stages.

– **Allows for Change:**

Since the main focus of the agile process is the satisfaction of the client, it allows for changes in plans. These changes may be introduced at any point of the development process.

– **Purpose to Team Members:**

Teams with purpose are always more productive, members challenge themselves to do more and be more efficient. Therefore, the agile process focuses on giving a shared sense of ownership and goals for the team members.

1.3.2 The Adopted Method : Scrum

The agile project management system Scrum establishes its objective to generate high-quality products through its ability to adapt projects to client need changes. A Scrum framework uses "Sprints" as work units that run between one and four weeks to allow development teams work on selected "Product Backlog" prioritized items. Throughout every Sprint the development team generates operable product versions through the production of an "Increment". Scrum exists to provide immediate client value through regular assessments and persistent enhancement of development procedures.



Figure 1.4: Scrum Process

Conclusion

The current system has shown limitations in terms of performance, flexibility, and ease of maintenance. Manual tasks, poor collaboration between teams, and lack of adaptability to changing workloads have made it difficult to meet business needs effectively.

By switching to a modern DevOps approach, we aim to overcome these issues. The new solution improves teamwork, speeds up delivery, and ensures the system runs reliably in all situations.

This transformation is not just about using new tools, but about adopting a more efficient and automated way of working. In the next chapter, we will explain how each DevOps and cloud technology contributes to building and running this improved system.

Chapter 2

DevOps Fundamentals and Tooling

Introduction

Today's software teams are expected to deliver new features quickly, fix bugs rapidly, and keep the system stable. DevOps is a culture and set of tools that aids in bringing the two teams together - Development and Operations. It focuses on automation, continuous integration and better collaboration.

DevOps uses automation, monitoring, infrastructure as code (IaC), containers, and scalable deployment tools to solve challenges of slow releases, poor communication, and manual work.

In this chapter we will examine some of the important concepts of DevOps such as CI/CD, container orchestration, cloud computing, monitoring. We will compare some tools and illustrate why certain of them is leveraged in this project by showing how they fit in for a complete DevOps pipeline.

2.1 DevOps Overview

2.1.1 Definition of DevOps

DevOps is the combination of two words – Development Operations. It both a culture and a set of practices which intends to improve software development, testing, delivery, and maintenance. DevOps breaks down the silos between developers and operations and encourages the two groups to work closely together. In other words, they are involved together throughout the lifecycle of software development. From writing the code to running it, the two functions work hand in hand.

DevOps aims to increase software delivery speed while enhancing the stability, quality, and security of applications. We can accomplish this through automation, CI/CD systems, live monitoring, and feedback loops. DevOps enables teams to react rapidly to user requests, fix bugs more rapidly, and launch new features more frequently.

At its core, DevOps is based on a few important principles:

- **Collaboration** – Developers, testers, and system admins work as one team.
- **Automation** – Repetitive tasks like testing, building, and deployment are automated.
- **Continuous Feedback** – Teams use monitoring and logging to get feedback and improve.

- **Shared Responsibility** – Everyone is responsible for both the development and the performance of the application.

In short, DevOps helps create a fast, reliable, and efficient development process by combining people, processes, and tools in a smart way [1].

2.1.2 DevOps Toolchain

- **Plan:** This is the first stage where teams develop projects after collecting requirements and setting goals to determine construction methods.

- **Code:** The developers collaborate through version control tools with code management options like Git to achieve smooth teamwork.

- **Build:** Code transformation into executable code begins after developers complete their writing of program code.

- **Test:** The Test phase includes automated tests which verify the code operates as intended and tries detecting any potential errors.

- **Deploy:** The software is released through a continuous deployment (CD) pipeline, allowing frequent updates and new features to go to production servers, to the customer.

- **Operate:** The operations team leads software execution inside real environments to verify its functionality for end users.

- **Observe and Monitor:** System monitoring tools enable teams to observe performance while detecting issues so problems can be resolved efficiently [2].

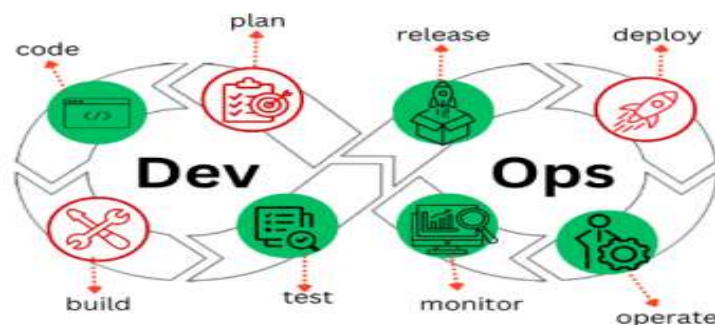


Figure 2.1: DevOps Workflow

2.2 DevOps Concepts and Tool Comparisons

2.2.1 Continuous Integration / Continuous Deployment (CI/CD)

Developers implement Continuous Integration (CI) by merging their code into a shared repository and usually do this several times a day. Each time, the code gets tested and

built so a bug does not happen. It boosts the quality of the software while accelerating its development.

Continuous Deployment (CD) is the next step, automatically releasing the code to staging or production after testing without manual steps. Faster, more reliable releases and rapid feedback from users are enabled [3].

CI/CD helps delivery of software easier and faster. In this project, we will compare and use popular CI tools which include Jenkins, GitLab CI and GitHub Actions. For CD we will use a GitOps approach which we will see later.

Separating CI and CD tools is an industry best DevOps practice as it increases visibility, ensures better tracking, and enhances control over the release process in production.

Table 2.1: CI Tools Comparison

Feature	Jenkins	GitLab CI	GitHub Actions
Integration	Highly flexible with many tools	Directly integrated with GitLab	Integrated with GitHub
Ease of Setup	Requires manual setup	Easy within GitLab	Easy within GitHub
Scalability	Highly scalable	Good within GitLab	Scalable via runners
Customization	High flexibility via plugins	Less customizable	Good with Actions workflows
Cost	Open-source and completely free	Free (with limits)	Free (with limits)

Selected Tool: Jenkins

We chose Jenkins because it is a completely free, open source automation server. Thanks to its flexibility, extensive plugin ecosystem, and adoption across the industry, Jenkins is very commonly used in building a CI pipeline. Setup and maintenance can be more complex but integrating with various tools and having an active community support is a big advantage. Jenkins is an open-source tool that does not have any licensing fee attached to it. Thus it is cost-effective, easy to scale, and not restricted to certain teams.

2.2.2 Virtualization and Containerization

Virtualization

Single computers can execute multiple virtual machines due to the virtualization technology. Every virtual machine operates independently like a standalone system containing its operating system and software collection.

The hypervisor functions as special software to develop and manage virtual machines. A physical computer operates multiple operating systems simultaneously thanks to this technology which ensures proper system isolation. Two examples of hypervisor software exist in the market such as VMware and VirtualBox.

The practice of virtualization extracts maximum efficiency from one computer system while reducing the need for multiple ones.

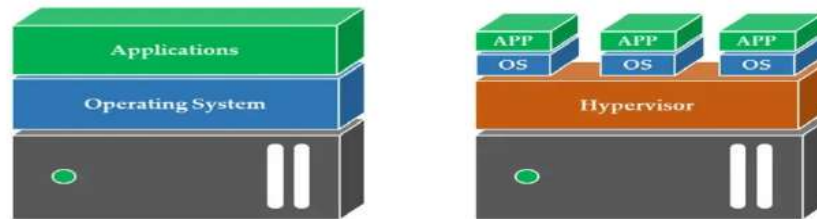


Figure 2.2: Bare Metal vs Virtualization

Containerization

Containerization permits an application to run in isolated environments aka containers which is a light-weight form of virtualization. Virtual machines use hypervisor to virtualize guest OS. Hence, they are heavier than containers. Container only packages and virtualizes app library. Containers bundle the application code and its dependencies so they will work uniformly no matter where the container is deployed. With this, it can provide key benefits such as faster startup times, better resource utilization and isolation.

Selected Tool: Docker

Docker, Podman and LXC are some of the more common tools. Docker has an advantage because it is simple to use and is well documented and has a huge community. Although Podman is more secure by design, it is still becoming popular. LXC offers containerization at the system level, which is more complex. So we will be using docker for our project.

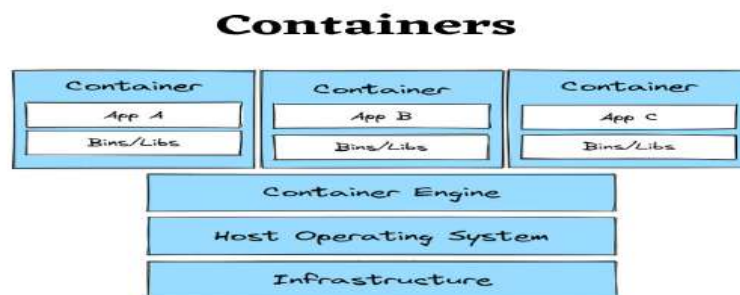


Figure 2.3: Containerization

Virtualization vs Containerization

Below is a comprehensive comparison table that highlights the key differences between virtualization and containerization to help better understand how these two approaches to computing differ:

Hybred Solution

A hybrid solution of virtualization and containerization uses both technologies together. VMs provide security and isolation, while containers are lightweight and fast.

Table 2.2: Comparison of Virtualization and Containerization

Aspect	Virtualization	Containerization
Technology	Hypervisors create VMs	Containers share host OS kernel
Overhead	High (full OS per VM)	Low (shared kernel)
Performance	Slower (VM overhead)	Faster (minimal overhead)
Isolation	Strong (separate OS for each VM)	Moderate (shared kernel)
Resource Use	Less efficient (more resources)	More efficient (shared resources)
Startup Time	Slower (OS boot)	Faster (seconds)
Portability	Limited (depends on hypervisor)	High (Docker-compatible systems)

In this setup, VMs run as the base, each with its own operating system. Containers run inside the VMs, sharing the same OS, which makes them efficient and easy to scale.

This approach is great for security, flexibility, and performance. This combination helps businesses optimize performance and resource usage while keeping applications secure and scalable.

The figure below illustrates what we have seen so far.

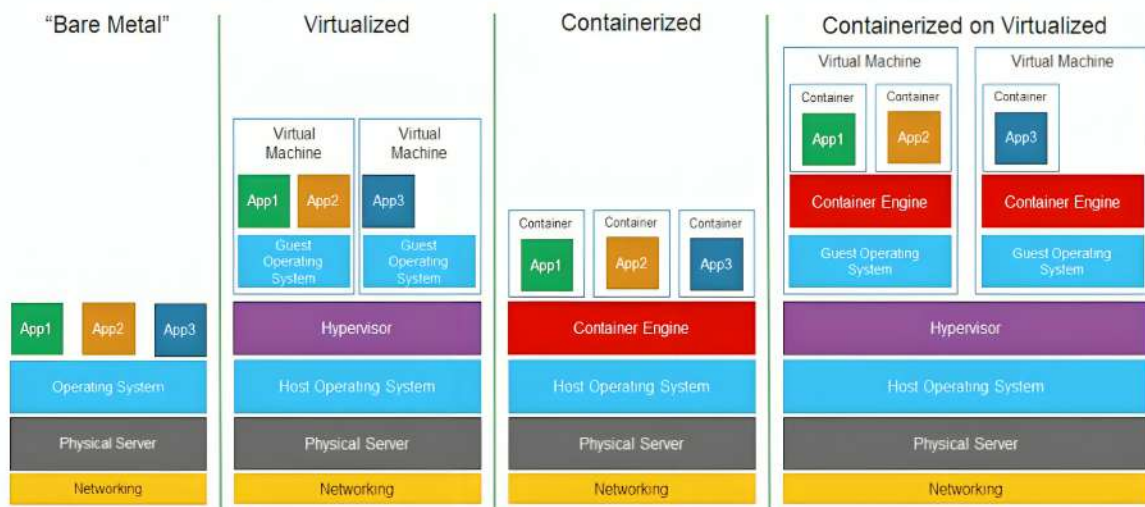


Figure 2.4: Hybrid Solution

2.2.3 Container Orchestration

Once you have containerized your application using Docker, the next step is to run it in production. Real-world applications usually do not run in isolation. They depend upon other services such as databases, message brokers, or backend APIs, and all could run in separate containers. As users increase, containers must scale to meet request level and guarantees availability performance. On the contrary, when demand is less, resources must be reduced to save cost and computing power. Handling all this dynamic scaling, service discovery, health checks, and inter-container communication manually is inefficient and full of errors.

This is where container orchestration comes in. The deployment, scaling, networking, and availability of containerized applications are automated. Important orchestration concepts include pods (groups of containers running together), scaling (automatically increasing and decreasing the number of containers based on demand), and rolling updates (updating application without downtime).

Many orchestration tools such as Kubernetes, Docker Swarm, and Apache Mesos (or HashiCorp Nomad). Docker Swarm is easier to start with as it is simple and has a Docker. Nomad is well-known for being easy to use and working with both container and non-container workloads. Because of its powerful features, large ecosystem, and backing from big cloud providers, Kubernetes is now the industry standard [4].

Selected Tool: Kubernetes

Kubernetes was selected for this project because of its powerful orchestration capabilities, active community, and seamless support for auto-scaling, load balancing, self-healing, and automated deployments. Being open-source and widely adopted, Kubernetes ensures a production-grade, flexible, and cloud-native environment that perfectly aligns with the needs of a modern DevOps pipeline.

2.2.4 GitOps and Deployment Automation

We will implement the GitOps method to be part of our Continuous Deployment (CD) strategy. It's the practice of using Git as the source of truth for everything. GitOps enables the management and automation of application delivery using Git-based workflows, making deployment traceable, auditable and repeatable. Anyone who wants to make a change to the system must first commit to Git; it will then automatically and securely sync to the running environment.

The fundamental principles of GitOps are declarative, automated, versioned; whether deployment or infrastructure. This technique makes it easy for teams to work together, roll back codes, and make environments consistent by using an app that watches the Git database to see what changes were made, which it will implement in the Kubernetes cluster [5]. GitOps workflows support many tools the most famous being ArgoCD and Flux. Although Flux is simple and flexible due to CLI-based management, ArgoCD is more user-friendly and supports multi-application environments out-of-the-box and has a web UI.

Selected Tool: ArgoCD

For this project, we chose ArgoCD to execute Continuous Deployment with GitOps. ArgoCD is an open-source, Kubernetes-native tool that can continuously watch Git repositories and apply the desired state automatically. It is a great tool for managing and visualising Kubernetes deployments using GitOps. The ease of use, visual interface, and strong ecosystem support which comes with this tool are the reasons why it was selected.

2.2.5 Cloud

Cloud computing has changed how infrastructure is developed, deployed, and maintained. Cloud computing allows you to avoid depending on physical servers and data centres for your infrastructure. It is a software-based service you access in the internet cloud. Greater flexibility, scalability, and cost-efficiency are provided.

There are three main models of cloud computing:

- **Infrastructure as a Service (IaaS)**: often known as IaaS, is the cloud model that is the most fundamental. It gives virtual computing resources like servers, network and storage. Users can fully manage their operating systems and the application they deploy. For example, there is Amazon EC2, Microsoft Azure Virtual Machines, and Google Compute Engine.
- **Platform as a Service (PaaS)**: PaaS refers to the cloud and platform based implementation models of software development. Developers can create, test, and deploy programs without managing infrastructure. Examples are AWS Elastic Beanstalk, Azure App Services, Google App Engine.
- **Software as a Service (SaaS)**: Software as a Service uses the internet to deliver applications that are purchased on a subscription basis. The service provider will manage the infrastructure, maintenance, and security. We may take example of Gmail, Microsoft Office 365 and Dropbox [6].

Several cloud providers dominate the market today:

- **Amazon Web Services (AWS)**: Amazon Web Services (AWS) is a widely-used cloud service. This cloud platform is the most mature one and has a huge range of services. It can easily change in size and is very reliable.
- **Microsoft Azure**: Microsoft Azure holds a high popularity quotient in enterprises using Microsoft technology. It offers strong integration with products like Active Directory, Windows Server, and Office 365.
- **Google Cloud Platform (GCP)**: Google Cloud Services (GCP) is known for having some great services and tools that can help in machine learning and data analytics. This is a good option for heavy data and AI work.

Selected Tool: AWS

The reason why AWS was chosen for this project is that it is the most popular cloud. It helps with development and other project practices. It's super scalable and reliable as an AWS service. Also, you can integrate a lot of tools like Jenkins, Docker, Kubernetes, and ArgoCD easily with it. Furthermore, its generous free tier makes it a good fit for development, testing and deployment at an affordable cost.

2.2.6 Infrastructure as Code (IaC)

Infrastructure as Code is one of the vital concepts of DevOps that allows provisioning and management of infrastructure by code instead of manually. IaC is the practice of managing infrastructure through the definition of machine-readable files; i.e. code.

Benefits of IaC include:

- **Automation**: Infrastructure setup and updates can be automated for minimal human intervention and human errors.
- **Reproducibility**: Reusability of the same configuration files leads to the creation of the same environment for development, test, and production. This gains us so much time comparing to manually provisioning these environments

- **Version Control:** Version control for infrastructure as code is very similar to application code. Because of this, we can store it in Git to track changes and rollback. Thus, all the information gets documented so that team members can work together correctly [7].

Comparison of IaC tools:

- **Terraform:** Open-source, cloud-agnostic tool that uses a declarative language (HCL) and supports multiple cloud providers.
- **AWS CloudFormation:** AWS-native service for provisioning resources using YAML or JSON templates.
- **Pulumi:** Allows defining infrastructure using general-purpose programming languages like Python, JavaScript, or Go.

Selected Tool: Terraform

Terraform was chosen because of its flexibility, multi-cloud support, and strong community. Terraform is an open-source tool that is totally free to use and allows you to define infrastructure as code.

2.2.7 Monitoring and Observability

Monitoring and observability are essential for maintaining system health. Observability means using data, like logs, metrics, and traces, to better understand what is going on inside a system. It allows us to see the system's behavior, making it easier to detect and fix any problems that arise.

Monitoring is the activity of observing key metrics and other activities such as CPU usage, response time, and more. It detects problems quickly using alerts when a deviation occurs.

The combination of observability and monitoring enables teams to get better control over the systems.

Why metrics and alerts are important

- **Metrics:** Provide quantitative data such as CPU usage, memory consumption, and request latency, which help in tracking system performance over time.
- **Alerts:** Notify teams in real time when specific thresholds or anomalies are detected, enabling quick responses to potential incidents.

Tools Comparison

- **Prometheus + Grafana:** A popular open-source stack where Prometheus collects metrics and Grafana visualizes them through interactive dashboards.
- **ELK Stack (Elasticsearch, Logstash, Kibana):** Focuses more on log aggregation, search, and visualization.
- **Datadog:** A commercial, all-in-one observability platform with integrated metrics, logs, and traces.

Selected Tools: Prometheus + Grafana

Prometheus and Grafana were chosen for the project because they are open-source, powerful, and have community support. For gathering metrics for the timing series, Prometheus is great, while the Grafana provides beautiful utilization dashboards to specify the performance and health of the system.

2.3 Selected Tools Overview

2.3.1 Docker

Docker is a free platform that lets you package an application into a container. This container has everything you need to run the application. This includes the code, runtime, tools, libraries, dependencies, etc. the app works the same, regardless of whether it's running on the developer's laptop or the production server. Docker is designed to solve the "It works on my machine" problem. Thus, it manages to make the developers and operations teams work in a consistent manner with significantly less friction

Use Cases of Docker

Docker provides significant advantages in various stages of the software development lifecycle:

- **Development and Testing:** Developers can create isolated and consistent environments on any system. This ensures uniformity across team members' setups and reduces environment-related bugs.
- **Application Deployment:** Docker containers encapsulate the application and its dependencies, making deployment fast, reproducible, and environment-agnostic.
- **Microservices Architecture:** Each microservice can run in its own container, allowing for independent development, scaling, and maintenance of different parts of the application.
- **Multi-Cloud and Hybrid Cloud Environments:** Docker increases application portability, making it easier to move between different cloud providers like AWS, Azure, or on-premise infrastructure.
- **CI/CD Integration:** Docker integrates seamlessly with CI/CD pipelines, enabling efficient automation of builds, testing, and deployments with high repeatability.

Advantages of Docker

Docker introduces several benefits that make it a preferred tool in DevOps workflows:

- **Portability:** Containers can run on any system with Docker installed, regardless of the underlying infrastructure or operating system.
- **Rapid Deployment:** Containers are lightweight and start almost instantly, enabling faster testing, development, and scaling of applications.
- **Resource Efficiency:** Docker containers consume fewer resources compared to traditional virtual machines, allowing better utilization of system hardware.

- **Increased Developer Productivity:** Docker empowers developers to focus more on coding and less on environment management, accelerating the software development lifecycle.
- **Cost Efficiency:** By optimizing infrastructure usage and streamlining development workflows, Docker can significantly reduce operational costs.
- **Enhanced CI/CD:** Docker enhances pipeline automation, enabling smoother and more reliable build, test, and deploy stages.
- **Community and Ecosystem:** Docker boasts a large and active community, rich documentation, and a wide ecosystem of tools and extensions available via Docker Hub [8].

Docker Hub : is a registry service that is offered by docker. It helps users easily store and share container images. Developers can pull ready-to-use images from Docker Hub to save time and avoid starting from scratch. It also allows users to push their own images so that teams can save their containers and reuse them. Docker Hub provides thousands of public images and the possibility of keeping private images.

2.3.2 Jenkins

Jenkins is an open-source automation server that enables Continuous Integration (CI). In this project, we use Jenkins for implementing a CI pipeline that builds and tests the application automatically whenever the code base gets updated.

Jenkins is widely used in many DevOps workflows. Some common use cases include:

- **Continuous Integration:** Automatically building and testing code changes.
- **Automated Testing:** Running unit, integration, and performance tests as part of the pipeline.
- **Build Automation:** Compiling code, packaging applications, and generating artifacts.
- **Pipeline Orchestration:** Managing complex workflows that include multiple steps and tools.
- **Notification and Reporting:** Notifying developers of build status and test results.

2.3.3 Kubernetes

Kubernetes, or K8s, is open-source software that automates the deployment, scaling and management of applications in containers. Originally, Google made Kubernetes. The Cloud Native Computing Foundation (CNCF) now develops and maintains it.

Kubernetes is the go-to container orchestrator that can manage applications in a reliable, consistent and scalable way. It manages how and where containers are deployed across a cluster of virtual machines, facilitating continuous development even on partial failure [9].

Kubernetes Fundamentals

Kubernetes provides a declarative approach to configuration using YAML files, enabling developers to specify how their applications should run. Its core functionalities include:

- **Desired state management:** You use YAML files to instruct Kubernetes regarding the number of containers (Pods) you want it running. Along with their image, how should it connect, and more. Kubernetes ensures this desired state is always met.
- **Automation:** Kubernetes automates tasks such as scheduling, health monitoring, and restarts.
- **Self-healing:** Kubernetes takes care of things when the enterprise software goes down or if the whole is down. It will replace anything which has gone down.
- **Scalability:** You can easily update the number of replicas in a Deployment file to scale your applications further up or down.
- **Portability:** Kubernetes can be used in multiple environments including your laptop (Minikube), public clouds (AWS, GCP, Azure) and private data centres [9].

Because of these, Kubernetes is ideal to run a production environment, especially with microservices or large applications.

Key Concepts

Some of the fundamental components in Kubernetes include:

- **Pod:** A pod is the smallest unit in k8s. It usually runs one container, but can run more if needed. Containers in the same pod all share the same IP and storage space. Kubernetes actually runs Pods when you deploy an app there.
- **Deployment:** A Deployment is that controller which manages Pods. It guarantees that a certain count of Pods are continuously operating. If one crashes, the Deployment creates a new one. Through deployments, you can apply rollouts and rollbacks as well.
- **ReplicaSet:** A ReplicaSet makes sure a certain number of Pods are always running. While Deployments manage ReplicaSets, you can also use ReplicaSet singularly, though this is not common. Deployments are preferred for higher-level control.
- **Services:** Services help because Pods come and go, changing their IPs. A Service provides a stable way to access a set of Pods. There are different types; **ClusterIP** Internal access only. **NodePort** exposes the service on each node's port. **LoadBalancer** opens the service to the web utilizing a load balancer from the cloud provider.
- **Ingress:** Ingress manages external access to the services in a Kubernetes cluster, typically HTTP/HTTPS traffic. It provides routing rules and allows you to expose multiple services under a single IP, offering more control over traffic routing.
- **Namespaces:** Namespaces help you organize objects within your cluster. For instance, having a separate "dev" namespace and "prod" namespace isolates the environments.
- **ConfigMaps and Secrets:** ConfigMaps help in saving non-sensitive data the application use, like URLs, file paths, etc. Secrets are like ConfigMaps but for storing private data like passwords, access tokens, and ssh keys [9].

Kubernetes Cluster Architecture

Kubernetes runs in a cluster made up of multiple machines (nodes). It follows a master-worker architecture, where some nodes manage the cluster (control plane), and others run the applications (worker nodes).

Control Plane (Master Node) : The control plane is the brain of the Kubernetes cluster. The control plane manages everything and makes critical decisions for cluster-wide behavior.

- **kube-apiserver:** is the interface for the Kubernetes control plane. This API server processes all commands and requests coming from users, CLI tools, or dashboards.
- **etcd:** It is a distributed key-value store that stores the cluster state and configuration. It's like the memory of Kubernetes.
- **kube-scheduler:** is responsible for choosing a worker node to run a pod.
- **controller-manager:** Runs different controllers that handle tasks like checking if a Pod should be restarted or if a Deployment needs more Pods.

Worker Nodes: Responsible for running containers.

- **kubelet:** on every worker node as a small agent. It talks to the control plane to make sure the containers are working well.
- **kube-proxy:** The program used to manage the network rules for Kubernetes Services.
- **Container Runtime:** The software for execution of containers is called Container Runtime. Examples include Docker and CRI-O [9].

This architecture allows Kubernetes to be scalable, resilient, and highly available.

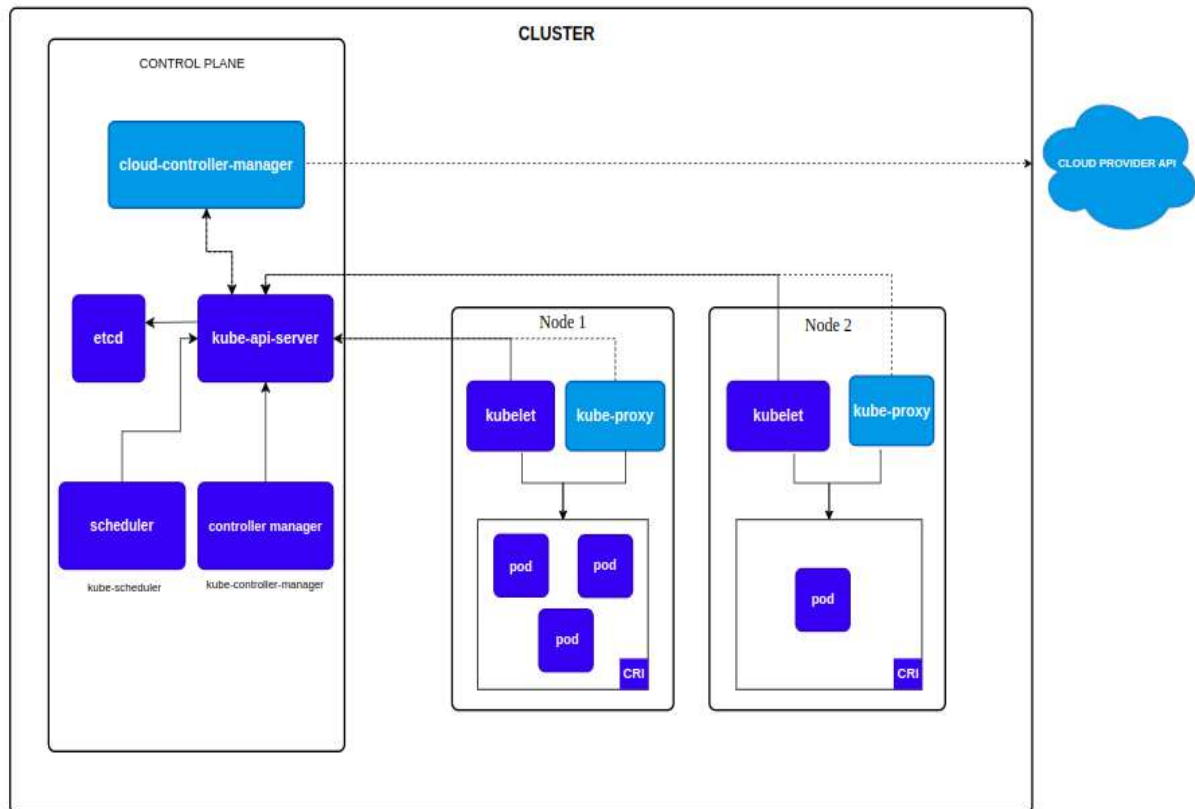


Figure 2.5: Kubernetes Architecture

2.3.4 ArgoCD

ArgoCD is a Continuous Delivery tool for Kubernetes, and it is declarative and GitOps-based. It is part of Argo Project, a CNCF (Cloud Native Computing Foundation) project. ArgoCD is a tool that automates the management and synchronization of Kubernetes resources based on Git changes.

Instead of using a push-based deployment strategy, ArgoCD uses a pull-based approach where it pulls the application configuration from a Git repository and applies it in the target Kubernetes cluster. With the help of this GitOps model, infrastructure and application tracing, stability and simple connectivity are improved.



Figure 2.6: GitOps deployment approach with ArgoCD

Main Features of ArgoCD

- **Git as the Source of Truth:** With ArgoCD, the Git repository has all Kubernetes configuration (manifests). So, your Git repo is the "single source of truth" for your deployments. Argo CD keeps an eye to your Git repo for changes, making sure that the deployed state of your application and the Git state is in sync. Fewer mistakes and greater uniformity across setups.
- **Declarative Application Management:** DevOps engineers describe the desired state of their applications with YAML manifests or Helm/Kustomize templates. ArgoCD always compares this desired state to the actual state in the cluster and alerts or acts when there is a mismatch. This makes deployments predictable and reproducible.
- **Web UI, CLI, and API Access:** ArgoCD comes with a web UI interface that allows for easy access and management of clusters. visualize deployment. Besides, you can access the REST and command-line interface `argocd`. API for scripting and automation. The user interface shows the sync status, differ-. You have the option to rollback if there are differences.
- **Automatic Sync and Health Checks:** ArgoCD can sync the application without requiring user intervention. Automatically, when it observes a change in Git. It also checks the health of Kubernetes. Resources to make sure it is running fine. This allows teams to automate the full. deployment cycle [10].

Advantages in CI/CD Automation

Using ArgoCD in CI/CD pipelines enhances the delivery process by introducing GitOps principles in the CD phase:

- **Separation of CI and CD:** CI tools such as Jenkins or GitHub Actions focus on building, testing, and pushing container images. ArgoCD manages the deployment by monitoring Git repositories and synchronizing updates with the Kubernetes cluster. It helps with maintainability and security issues.
- **Improved Security:** As ArgoCD is pull-based, it doesn't require external CI/CD tools to have direct access to the Kubernetes cluster. ArgoCD fetches modifications from Git and implements them within the cluster, minimizing attack surfaces.
- **Safe and Predictable Deployments:** Every deployment is tracked through Git. Rollbacks are as simple as reverting a commit. ArgoCD checks that the live system matches the state defined in Git.
- **Real-Time Monitoring:** The UI lets DevOps engineers and developers know exactly what is deployed, and how it has changed. It shows variations between the Git and cluster state and gives the option to deny or approve syncs.
- **Scalability and Team Collaboration:** ArgoCD can be used effectively by many teams trading in multiple applications. Every team manages its apps and Git repos without affecting the others [10].

2.3.5 Amazon Web Services (AWS)

Amazon Web Services (AWS) is one of the most popular cloud platforms used by startups, large companies, and even governments. It offers flexible pricing—you only pay

for what you use. This makes it affordable whether you're spending a few cents or much more.

Flexible Pricing and Cost Control

With AWS, there are no big upfront costs. You can start small and grow as needed. Companies can pay only for what they use [11].

Scalability and Agility

AWS makes it easy to scale your resources up or down. Using EC2 (for computing) and S3 (for storage) can automatically adapt to demand, allowing your application to run smoothly [11].

Innovation with New Technologies

AWS is equipped with cutting-edge tools like AI, machine learning and IoT. For instance, AWS Lambda runs your code without requiring server management, while SageMaker aids in building ML models [11].

Strong Security and Compliance

AWS has strong security features to protect your data. Using IAM (Identity and Access Management) gives you the power to monitor who gets access to what. IAM solutions help you meet global security standards [11].

2.3.6 Terraform

Terraform is a tool developed by HashiCorp that enables the management of cloud infrastructure using code. You define your resources in files instead of setting resources manually so that it is easy to automate resources across any cloud platforms AWS, Azure, Google Cloud, etc.

Terraform is powerful tool that can help you manage your infrastructure automatically and at scale. It is an important part of today's DevOps practices and is widely implemented by teams building reliable and repeatable cloud environments.

Terraform Workflow

Terraform has a simple and clear workflow consisting of three steps:

- **Write:** Using configuration files (in .tf format) you define your infrastructure. These could be VMs, storage, networks, security groups, load balancer, etc. These resources can come from one or multiple providers.
- **Plan:** Terraform will verify your settings and display what it will do, detailing what it will add, change or remove. This will prevent surprises and allow teams to review them before it actually happens.
- **Apply:** When you apply a plan, Terraform will make the changes in the right order, respecting dependencies between resources. For instance, a VPC will be created before launching virtual machines inside it [12].

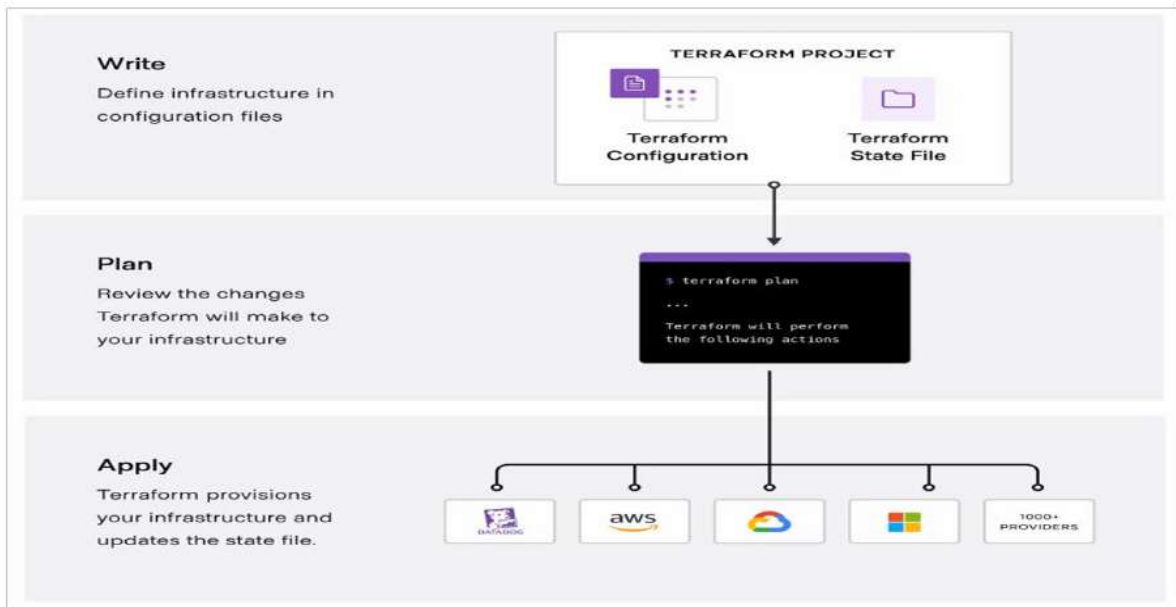


Figure 2.7: Terraform workflow

2.3.7 Prometheus

Prometheus is an open-source monitoring and alerting system that is designed for cloud-native environments. It is designed to collect, process and re store real-time metrics from different systems and applications. Prometheus is widely used in DevOps workflows. It helps ensure services are available and performant by enabling teams to observe their systems behaviour over time.

Prometheus is composed of several key components that work together to offer complete monitoring capabilities:

- **Prometheus Server:** The core component that scrapes metrics from configured targets at specified intervals and stores them locally.
- **Time-Series Database (TSDB):** A highly efficient storage engine used to store collected metrics in a format optimized for time-series queries.
- **PromQL:** A query language used to extract, filter, and aggregate data from the TSDB.
- **Alertmanager:** Responsible for handling alerts generated by Prometheus rules. It supports deduplication, grouping, and routing to various notification channels like email, Slack, or PagerDuty.
- **Exporters:** Lightweight components or agents that expose metrics from third-party systems like databases, message queues, or web servers in a format Prometheus understands.
- **Pushgateway:** Allows ephemeral or batch jobs to push metrics to Prometheus when scraping is not feasible.
- **Web UI:** A built-in graphical interface used to run PromQL queries, view targets and alerts, and explore metrics data [13].

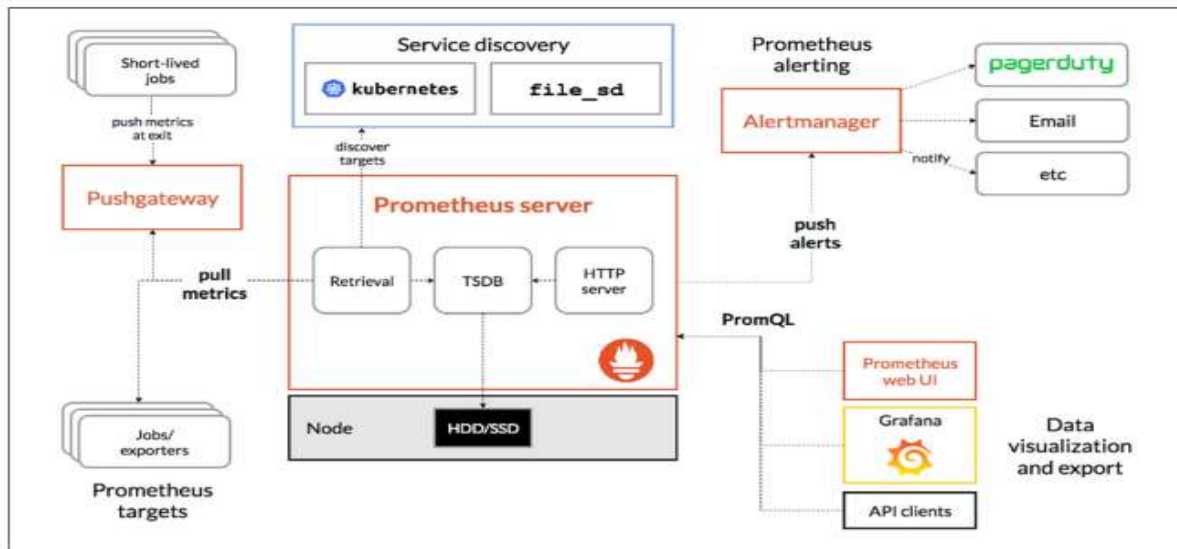


Figure 2.8: Prometheus Architecture

2.3.8 Grafana

Grafana is a free and open-source tool used to view and analyze data. It connects to many data sources like databases, cloud services, and monitoring tools. With Grafana, we can create custom dashboards to show data in charts, tables, and graphs.

It also supports real-time alerts and lets us filter or group data as needed. Many teams use Grafana to monitor systems, track app performance, and get live updates. Its easy interface and support for many sources make it great for keeping an eye on everything in one place [14].

Conclusion

In this chapter, we explored how DevOps works and why it is essential in software development these days. We talked about important concepts such as CI/CD, containerization, orchestration, IAC, cloud computing and GitOps. Lastly, we presented an overview of the tools we have selected for the project explaining how they support these practices.

Chapter 3

Planning

Introduction

The essential first step in our work involves understanding and documenting client requirements. A failure to comprehend requirements correctly will produce software application that can miss customer expectations. In this upcoming section we will deep dive into of details project requirement specification alongside actor recognition and strategic planning for project steps. We will establish the product backlog and organize sprint planning to maintain a controlled development process.

3.1 Requirement Analysis

In this part, we will identify the actors and deep dive into the functional and non-functional requirements that this application must fulfil.

3.1.1 Identifying Actors

- **Developer:** Application development and maintenance duties belong to the Developer role. The major responsibility of Developers includes pushing their code into the repository.

- **DevOps Engineer:** To achieve automatic code integration testing and deployment, the DevOps Engineer implements pipeline automation for CI and CD operations. The DevOps Engineer maintains infrastructure while monitoring system performance to guarantee scalability and reliability.

3.1.2 Functional Requirements

Integration with GitHub Source Code Repository: The application source code must be controlled via a private GitHub source code repository. Every time a new commit occurs, the CI process will automatically be triggered by Jenkins. This guarantees that any change receives a real-time test with validation.

Continuous Integration with Jenkins and SonarQube CI process refers to building and testing application via Jenkins. SonarQube analyzes the quality and coverage of code.

Storing Build Artifacts in Nexus Once the build succeed, jenkins must save the application artifacts (ex: jar files) in Nexus. This aids in the tracking and controlling of versions of builds which will be used later during image creation and deployment.

Containerizing the Application with Docker Every app component must have its own Dockerfile. During the CI process, you must build Docker images, and push to Docker Hub. Kubernetes will use the images for deployment later on.

Provisioning Infrastructure with Terraform The desired AWS cloud infrastructure must be provisioned automatically using Terraform.

Deploying to Kubernetes Using ArgoCD Once the Docker images are built and pushed, the CD pipeline will be managed by ArgoCD. It will monitor our GitOps repository containing K8s manifests and automatically sync changes to the K8s cluster. This guarantees that no manual work is required for deploying the new version.

Monitoring the System with Prometheus and Grafana Prometheus is capable of gathering metrics like CPU, memory and pod status from the K8s cluster. Dashboards in Grafana will make data more visually attractive.

3.1.3 Non-Functional Requirements

Availability Ensuring high availability is a top priority for this solution. We understand the importance of having a system that is always operational for users. The infrastructure is designed to be resilient, capable of handling potential failures without impacting the availability of the service.

Performance Performance is a key component of the solution. We are committed to providing efficient and fast operation, meeting user expectations for quick response times and a seamless user experience.

Scalability The system and infrastructure must be capable of scaling to handle an increase in traffic and users. The Kubernetes cluster will automatically scale the application to ensure the system can manage growth effectively.

3.2 Requirement Specification

A more thorough study of the functional requirements of our system is essential. This section presents a more detailed specification of these requirements based on the concepts of UML modelling, namely the use case diagram.

3.2.1 General Definitions and Tools

- **Unified Modelling Language (UML):** UML is a standardized modelling language enabling developers to specify, visualize, construct and document artifacts of a software system.

- **Use Case Diagram:** A use case diagram is a dynamic or behaviour diagram in UML. Use case diagrams model the functionality of a system using actors and use cases. Use cases are a set of actions, services and functions that the system has to provide.

- **Draw.io:** We used Draw.io, an online tool for drawing schemas and diagrams.

3.2.2 Global Use Case Diagram

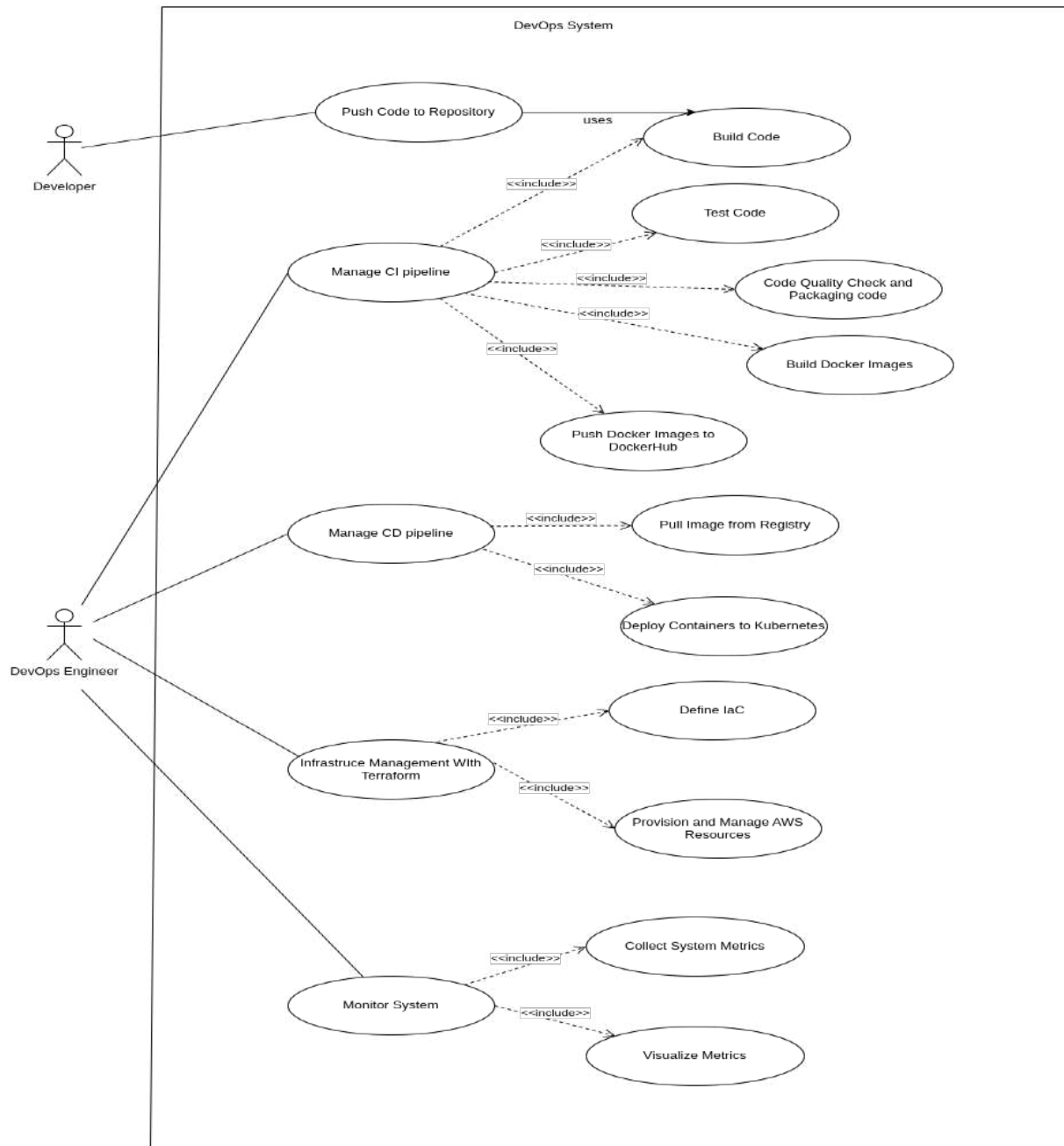


Figure 3.1: use case diagram

3.3 Project Planning with Scrum

This section analyzes the product backlog structure as a crucial Scrum component for feature prioritization and planning. Under Scrum methodology the team comprises various important roles. The team consists of these members:

- **The Scrum Master (Miss Malek Aribi):** The Scrum Master maintains Scrum methodology standards to unlock maximum team member performance through obstacle removal and disruption protection alongside Scrum principle implementation.

- **The Product Owner (Miss Malek Aribi):** The product ownership role of **Malek Aribi** involves achieving the highest possible value from the developed product. She maintains control over the product backlog through prioritization while making sure user stories have clear descriptions for effective implementation.

- **Development Team:** The development team at our project consists of **Ahmed Ben Sassi**. The development team member **Ahmed Ben Sassi** has the responsibility to execute all tasks included in the sprint period. He operates autonomously to meet requirements while closely working with **Malek Aribi** to create high-quality product increments. **Ahmed Ben Sassi** maintains complete functionality in sprints while being present during planning reviews and retrospectives.

3.3.1 Product Backlog

The Product Backlog is an ordered list of every task that is known to be processed for the product. It is the single source of requirements for any changes to be made to the product. The needs are specified through user stories. A user story has the following format:

As a < type of user >, I want < some goal > so that < some reason >

After few meeting with Miss Asma BEN ABDALLAH, the following is the final product backlog:

Table 3.1: Full Product Backlog

ID	Task	Description
1	Create VPC and Subnets	Define and provision a custom VPC with public subnets for Jenkins, SonarQube, and Nexus instances.
2	Create Security Groups	Create Security Groups for the EC2 instances allowing necessary ports for each server.
3	Provision EC2 Instances	Provision 3 EC2 instances for Jenkins, SonarQube, and Nexus.
4	Install Required Jenkins Plugins	Install essential Jenkins plugins for CI pipeline.

ID	Task	Description
5	Configure Jenkins Tools	Set up JDK, Maven, Node.js, and sonar-scanner in Jenkins global tool configuration.
6	Set Up Jenkins System Settings	Configure Jenkins system settings to communicate with SonarQube server.
7	Configure SonarQube Token	Generate the SonarQube token.
8	Configure Jenkins Credentials	Add GitHub and SonarQube credentials in Jenkins.
9	Set Up Nexus Repository	Create the 'scrum-ui-artifacts' repository in Nexus to store build artifacts.
10	Configure Source Code	Configure source code of frontend and backend for sonar scanning and artifact upload to Nexus.
11	Setup CI pipeline	Setup Jenkins CI pipeline and make necessary configuration.
12	Write CI Pipeline Stages in Jenkinsfile	Define stages for build, test, SonarQube analysis, packaging, and artifact upload to Nexus.
13	Create Dockerfiles for scrum-ui and scrum-app	Write Dockerfiles to containerize frontend and backend.
14	Install Docker on Jenkins Server	Install Docker on Jenkins server to enable building and pushing images.
15	Install and Configure Docker Plugin in Jenkins	Install Docker plugin and configure Docker tools in Jenkins.
16	Create Docker Repositories on Docker Hub	Create repositories for 'scrum-ui' and 'scrum-app' on Docker Hub.
17	Configure Docker Credentials in Jenkins	Set up Docker Hub credentials in Jenkins.
18	Integrate Docker into Jenkins Pipeline	Modify Jenkinsfile to build and push Docker images.
19	Update VPC and Subnets	Add 2 private subnets across AZs to host Kubernetes cluster.
20	IAM Configuration	Create and attach IAM roles and policies for Cluster Autoscaler and AWS Load Balancer Controller.
21	Provision Bastion Node	Deploy a bastion host in public subnet for secure SSH to private instances.

ID	Task	Description
22	Provision Master Node	Launch a master node (control plane) in private subnet for Kubernetes cluster.
23	Provision Launch Template and ASG	Create a launch template and Auto Scaling Group for Kubernetes worker nodes.
24	Automate Worker Node Patching	Write and schedule crontab script to patch worker nodes with labels.
25	K8s Manifest Files	Prepare and create Kubernetes manifest files for the application.
26	Prepare GitOps Repository	Create GitOps repository with all Kubernetes manifest files.
27	Update Docker Version in GitOps Repository	Keep GitOps updated with new Docker image versions.
28	Email Notification	Configure email notifications for CI pipeline status.
29	GitOps Setup with ArgoCD	Install and configure ArgoCD to sync GitOps repo and deploy application.
30	Autoscaling Demo	Simulate load and observe HPA and CA scaling behavior.
31	Grafana Dashboard	Access Grafana dashboard for real-time cluster performance and resource monitoring.

3.3.2 Sprints Planning

Scrum functions by allowing Agile teams to finish work assignments through designated time periods known as sprints. Our project has 5 sprints.

The schedule for each sprint appears in the table which runs for four weeks while targeting dedicated work items

Table 3.2: Summary of the Four Sprints

Sprint	Title	Brief Description
1	Setting Up Infrastructure as Code (IaC) and Automation Tools	In this sprint, the focus is on provisioning the cloud infrastructure, including VPC, subnets, EC2 instances and configuring tools such as Jenkins, SonarQube, and Nexus.
2	Implementing CI Pipeline and Docker Integration	This sprint covers building a full CI pipeline with Jenkins, writing the pipeline stages, containerizing the applications using Docker, and pushing images to Docker Hub.
3	Designing a Highly Available and Auto-Scaling Kubernetes Cluster	This sprint includes setting up a production-ready Kubernetes cluster using private subnets, configuring IAM roles, bastion host, control plane, worker nodes, and writing Kubernetes manifest files.
4	Deployment to Insight: GitOps with ArgoCD and Real-Time Observability	The final sprint focuses on setting up GitOps with ArgoCD for automated deployment, configuring email notifications, demonstrating autoscaling, and integrating monitoring using Grafana.

3.4 Proposed Architecture

3.4.1 AWS Architecture

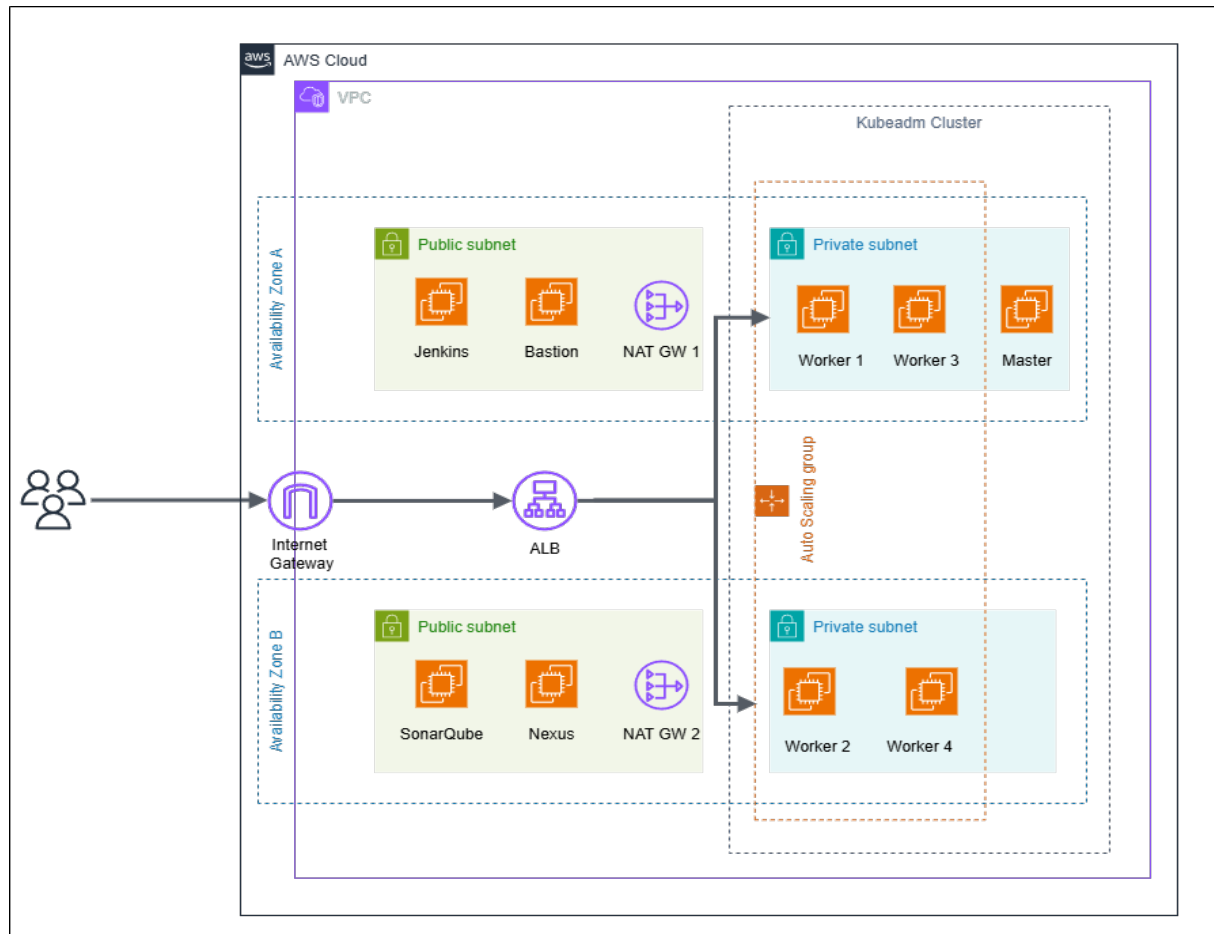


Figure 3.2: AWS Architecture

We built our system inside a Virtual Private Cloud (VPC), which gave us a safe and private network inside AWS. This helped us control how different parts of our system talked to each other and kept everything secure.

To ensure high availability and fault tolerance, we deployed the infrastructure across two separate Availability Zones (AZ1 and AZ2). These zones are distinct physical data centers within the same region. By spreading our components across multiple AZs, we made the system resilient against failures in any single zone, which improved the reliability of our application.

In each availability zone, we created two types of subnets: public and private. The public subnets were for components that needed to connect to the internet. We put Jenkins here to handle automated tasks like building code and running tests. We also placed SonarQube in the public subnet to check our code quality and find errors early. The Nexus repository was also here, which stored our artifacts.

To securely access resources in the private subnets, we used a Bastion host as a secure gateway. This host allowed authorized users to connect via SSH to internal servers without exposing those servers directly to the internet.

The private subnets contained critical infrastructure components that were not pub-

licly accessible. We ran our Kubernetes cluster, created with kubeadm, within these private subnets to increase security by limiting direct internet exposure. The cluster was supported by an Auto Scaling Group (ASG), which automatically adjusted the number of worker nodes based on workload demands. This autoscaling ensured that our cluster maintained optimal performance and cost-efficiency by dynamically matching resources to current needs.

Because the private subnets had no direct internet access for security reasons, we deployed NAT Gateways in each public subnet across both availability zones. These NAT Gateways allowed resources in the private subnets, in our case K8s cluster, to initiate outbound internet connections for tasks like downloading updates or pulling container images, without exposing them to inbound internet traffic.

To handle user traffic, we set up an Application Load Balancer (ALB). The ALB received all incoming requests from users and then forwarded this traffic to the Ingress controller inside our Kubernetes cluster. The Ingress managed how the traffic was routed to the different applications running in the cluster. This setup helped our app handle many users at the same time without slowing down, while keeping everything organized and secure.

In short, we designed our system to be secure, flexible, and easy to manage. Public tools like Jenkins and SonarQube were accessible but safe, while our main app ran in a private, protected network. We used autoscaling and load balancing to handle changing workloads, and NAT Gateways to allow secure internet access for our private servers.

3.4.2 DevOps Architecture

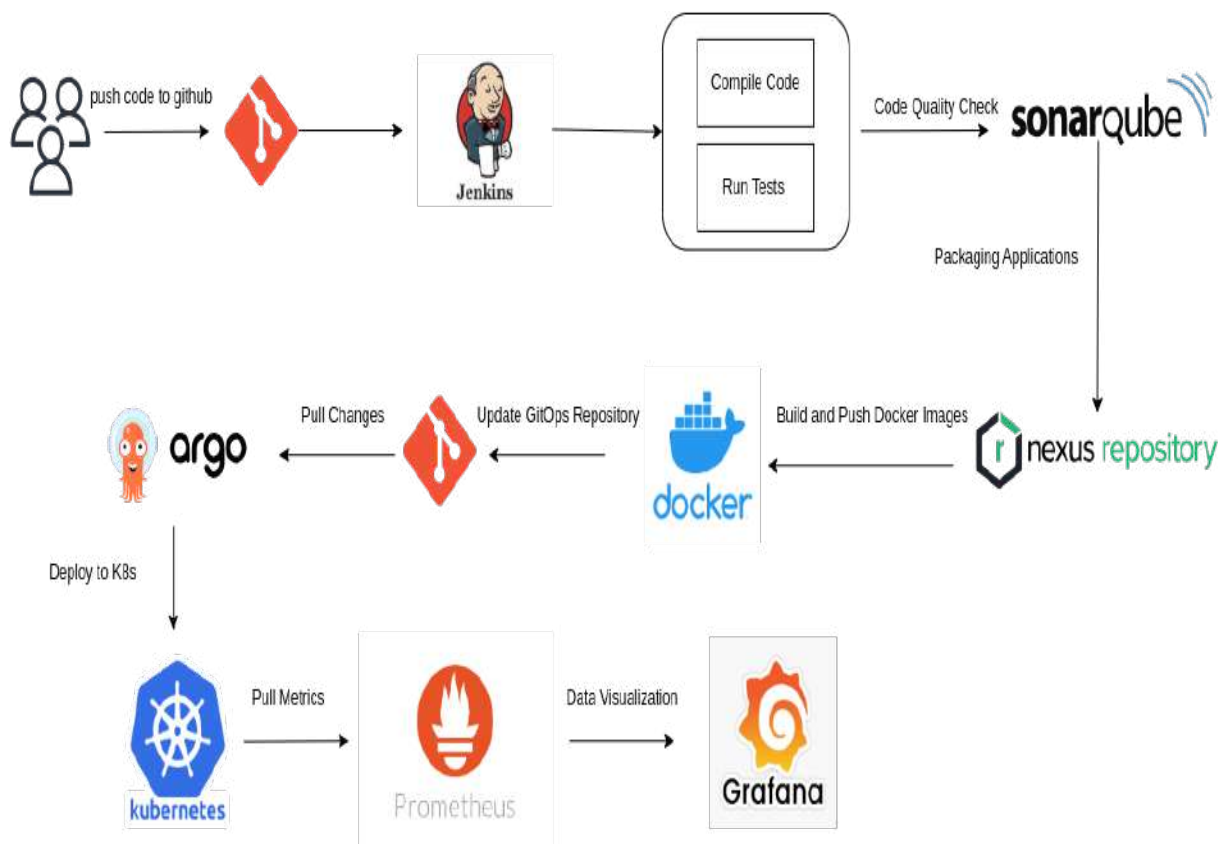


Figure 3.3: DevOps Architecture

The development team starts by pushing their code to a Git repository. Jenkins pulls the code, compiles it, runs tests, and checks the code quality using SonarQube. Next, Jenkins builds the app and uploads the build files to Nexus. Then, it creates a Docker image, pushes it to Docker Hub, and updates the GitOps repository with the new image version. ArgoCD watches this GitOps repo and automatically applies any changes to the Kubernetes cluster. To monitor the system, Prometheus collects data from the running apps and infrastructure. Grafana uses this data to show real-time dashboards, helping teams track performance and health.

3.4.3 Kubernetes Architecture

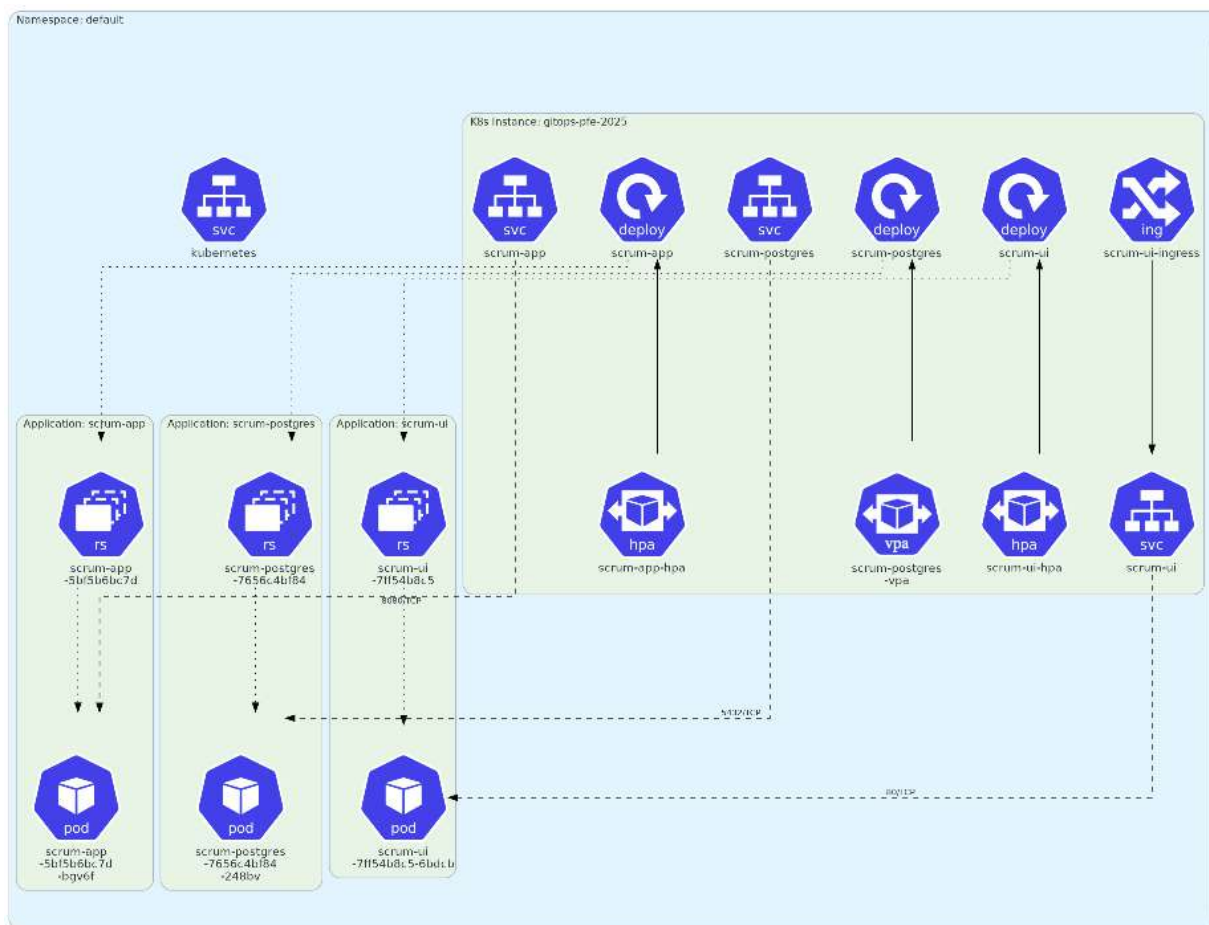


Figure 3.4: K8s Architecture

In our project, the three-tier web application was implemented and deployed in the default namespace on Kubernetes. The architecture separates the frontend (scrum-ui), the backend (scrum-app), and the database (scrum-postgres) clearly. We managed the application using Kubernetes components such as Deployments, ReplicaSets, Pods, Services, Ingress, and autoscaling for reliability and scalability purposes.

We built the scrum-ui as the frontend of our application, providing the user interface that interacted with the backend. We deployed it using a Deployment, which created a ReplicaSet and managed the lifecycle of the pods running our scrum-ui container.

To handle variable traffic loads, we added a Horizontal Pod Autoscaler (HPA) (scrum-ui-hpa) for the frontend. This allowed our application to automatically scale the number

of frontend pods based on CPU utilization, maintaining responsiveness and stability under high demand.

For external access to the frontend, we exposed the `scrum-ui` service using an Ingress resource. We integrated this with the AWS Load Balancer Controller, which automatically provisioned an Application Load Balancer (ALB) in our AWS environment.

When a user accesses our application through a web browser. The incoming HTTP request first reaches the AWS Application Load Balancer (ALB), which was automatically created and managed by the AWS Load Balancer Controller in response to our Kubernetes Ingress resource. The ALB uses target groups that point to the Kubernetes worker nodes, forwarding traffic to them based on listener rules defined by the Ingress resource.

Once the traffic reaches a worker node, the request is routed internally by `scrum-ui` service, which then forwards it to the corresponding `scrum-ui` pods running.

The `scrum-app` component served as our backend, processing logic and handling requests from the frontend. Like the frontend, we deployed it using a Deployment that managed ReplicaSets and Pods. We exposed the backend internally using a ClusterIP Service, allowing the frontend to communicate with it securely without exposing it externally.

We also implemented an HPA (`scrum-app-hpa`) for the backend to ensure it could automatically scale based on CPU load, which helped us handle spikes in traffic without degrading performance.

For our database `scrum-postgres` we also used a Deployment and ReplicaSet. The database was exposed internally via a ClusterIP Service, enabling secure communication with the backend.

Because PostgreSQL doesn't scale well horizontally, we used a Vertical Pod Autoscaler (VPA) (`scrum-postgres-vpa`). The VPA automatically adjusted the pod's CPU and memory based on usage, helping us manage resources efficiently without manual intervention.

Conclusion

In this chapter, we looked at what the project needs, we wrote down the requirements and showed how the system will work with a use case diagram. Then, we planned the work using scrum by creating a product backlog and organizing sprints. Lastly, we showed the main architecture for the cloud, DevOps, and Kubernetes to help understand what we will going to do in our project.

Chapter 4

Sprint 1: Settings Up Infrastructure as Code (IaC) and Automation Tools

Introduction

In Sprint 1, we focused on setting up the core tools needed for our DevOps journey. We started by provisioning and installing Jenkins for automation, SonarQube for checking code quality, and Nexus to store and manage our build artifacts. Once the servers were ready, we configured each tool to work smoothly together and support our future work. This setup prepared us for Sprint 2, where we will start building the CI pipeline and use Docker to containerize our applications.

4.1 Sprint 1 Backlog Product

Table 4.1: Sprint 1 Backlog Product

ID	Task	Description
1	Create VPC and Subnets	Define and provision a custom VPC with public subnets for the Jenkins, SonarQube, and Nexus instances.
2	Create Security Groups	Create Security Groups for the EC2 instances allowing necessary ports for each server.
3	Provision EC2 Instances	Provision 3 EC2 instances for Jenkins, SonarQube, and Nexus.
4	Install Required Jenkins Plugins	Install essential Jenkins plugins for CI pipeline.
5	Configure Jenkins Tools	Set up JDK, Maven, and nodejs, sonar-scanner in Jenkins global tool configuration.
6	Set Up Jenkins System Settings	Configure Jenkins system settings to communicate with SonarQube server.

ID	Task	Description
7	Configure SonarQube Token	Generate the SonarQube token.
8	Configure Jenkins Credentials	Add GitHub and SonarQube credentials in Jenkins.
9	Set Up Nexus Repository	Create the 'scrum-ui-artifacts' repository in Nexus to store build artifacts.
10	Configure Source Code	Configure source code of both frontend and backend to enable sonar scanning and uploading artifacts to nexus

4.2 Sprint 1 Realisation

- Create VPC and Subnets

We used Terraform AWS VPC module for our project, which simplifies and automates the process of creating the Virtual Private Cloud (VPC) and its related network components. Since we wanted to avoid re-writing the complete networking resources, we opted to use this official module as complete and ready-made.

We used CIDR block to define custom VPC with 2 public subnets across 2 availability zone for better availability. Using `data.aws-availability-zones.available` dynamically fetches the availability zones of the region and is not hard-coded.

We made an internet gateway (IGW) to enable our public instances to get the internet.

We do not have any private subnets in our infra so we disabled the NAT Gateway to save on cost and complexity. Last, we tagged our resources using the variables `var.tags` and `var.public-subnet-tags` for better management inside AWS.

All in all, the VPC module made it easy and helped save time for us .

```

You, 2 days ago | 1 author (You)
data "aws_availability_zones" "available" {
  state = "available"
}

You, 1 minute ago | 1 author (You)
module "demo-k8s-aws-vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.17.0"

  name = var.vpc_name
  cidr = var.cidr_block

  azs = [data.aws_availability_zones.available.names[0], data.aws_availability_zones.available.names[1]]
  public_subnets = [cidrsubnet(var.cidr_block, 8, 10), cidrsubnet(var.cidr_block, 8, 20)]

  create_igw = true
  enable_dns_hostnames = true

  # We are disabling NAT since there are no private subnets
  enable_nat_gateway = false
  create_private_nat_gateway_route = false

  tags = var.tags
  public_subnet_tags = var.public_subnet_tags
}

```

Figure 4.1: VPC

- **Provision EC2 Instances**

In this section of the configuration, we retrieve the latest version of the Ubuntu 22.04 (Jammy Jellyfish) amazon machine image (AMI) using the aws-ami data source. Through the use of filters, we controlled that the AMI was x86-64 architecture and from Canonical's official AWS account. This allowed us to launch our EC2 instance using the latest version of Ubuntu which comes with all the latest security fixes and performance enhancements.

```
***
data "aws_ami" "latest_ami" {
  most_recent = true

  ***
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-*server-*"] # Ubuntu 22.04 (Jammy Jellyfish)
  }

  ***
  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  owners = ["099720109477"] # Canonical's AWS account ID for Ubuntu AMIs
}
***
```

Figure 4.2: Ubuntu AMI

After fetching the ubuntu ami image, we then defined the EC2 instances that would be provisioned for the Jenkins, SonarQube, and Nexus services.

We created these instances using the same configuration and customized it for each service. We used the aws-ami.latest-ami data source to get the latest Ubuntu 22.04 AMI so that our instance will be provisioned using it. We used the t2.medium instance type to get a decent amount of resources for our services.

We launched each instance in the public subnet of our VPC. Also, we assigned security groups to control traffic. We made sure that each instance had public IP for accessibility. Also, we used the user data parameter to install the required services (Jenkins, SonarQube, and Nexus) on the instance automatically.


```
# Jenkins EC2 Instance
You, 55 minutes ago | 1 author (You)
resource "aws_instance" "jenkins" {
  ami                  = data.aws_ami.latest_ami.id
  instance_type        = "t2.medium"
  subnet_id            = module.demo-k8s-aws-vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
  key_name              = var.key_pair
  associate_public_ip_address = true

  You, 2 days ago | 1 author (You)
  root_block_device {
    volume_size = var.volume_size
  }

  user_data = file("../jenkins-installation.sh")

  You, 2 days ago | 1 author (You)
  tags = {
    Name = "jenkins"
  }
}
```

Figure 4.3: Jenkins EC2 Instance

```
# SonarQube EC2 Instance
You, 1 hour ago | 1 author (You)
resource "aws_instance" "sonarqube" {
  ami                  = data.aws_ami.latest_ami.id
  instance_type        = "t2.medium"
  subnet_id            = module.demo-k8s-aws-vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.sonarqube_sg.id]
  key_name              = var.key_pair
  associate_public_ip_address = true

  You, 2 days ago | 1 author (You)
  root_block_device {
    volume_size = var.volume_size
  }

  user_data = file("../sonarqube-installation.sh")

  You, 2 days ago | 1 author (You)
  tags = {
    Name = "sonarqube"
  }
}
```

Figure 4.4: SonarQube EC2 Instance


```
# Nexus EC2 Instance
...
resource "aws_instance" "nexus" {
  ami                = data.aws_ami.latest_ami.id
  instance_type      = "t2.medium"
  subnet_id          = module.demo-k8s-aws-vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.nexus_sg.id]
  key_name            = var.key_pair
  associate_public_ip_address = true

  ...

  root_block_device {
    volume_size = var.volume_size
  }

  user_data = file("../nexus-installation.sh")

  ...

  tags = {
    Name = "nexus"
  }
}
```

Figure 4.5: Nexus EC2 Instance

- **Configure Jenkins**

The screenshot proves that we were able to reach the Jenkins unlock page. This appeared immediately after installation. This page requested us to input the initial admin password which was on the server.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`/var/lib/jenkins/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

```
ubuntu@ip-172-32-10-24: $ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
ef77697919d24783863c59dca818b81c
ubuntu@ip-172-32-10-24: $
```

[Continue](#)

Figure 4.6: Unlock Jenkins

After we unlocked jenkins, we were taken to the plugin installation page. Jenkins presented us with a list of plugins that are commonly used. We selected the option to install suggested plugins to save time in setting up Jenkins for continuous integration and continuous deployment (CI/CD). Those included Git integration, pipeline support, credential management and more plugins.

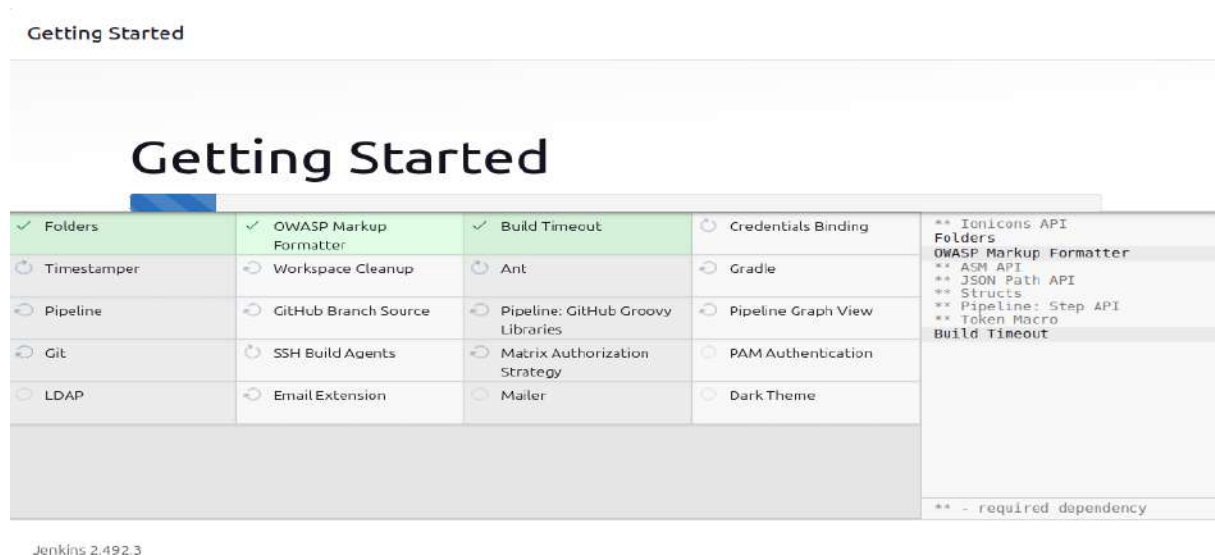


Figure 4.7: Plugins Suggestion

When the installation of the plugin finished, we set to create the first admin user. This was important step so that our Jenkins server is secure. We used the username admin and the password admin to create a simple user. With these login details, we successfully logged in and fully accessed the Jenkins interface. By using this user we were able to manage Jenkins settings, create pipelines and integrate with other tools in DevOps.



Figure 4.8: Create Admin User

And here is the jenkins dashboard!

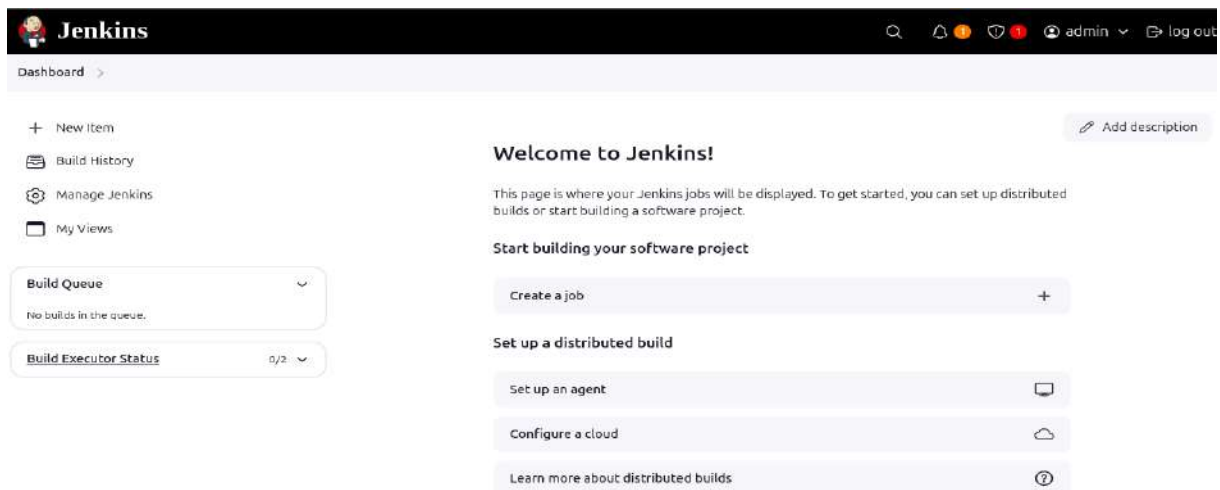


Figure 4.9: Jenkins Dashboard

We went to “Manage Jenkins” and then to “Manage Plugins” in the Jenkins dashboard to install plugins required for CI pipeline. Because of these plugins, Jenkins was able to integrate well with the tools we are going to use. We installed all necessary Jenkins Plugins which included Eclipse Temurin Installer Plugin, NodeJS Plugin, Maven Integration Plugin, SonarQube Scanner for Jenkins, Pipeline Stage View Plugin. This was necessary to prepare Jenkins for building, testing, analysing code, and visualising pipelines.

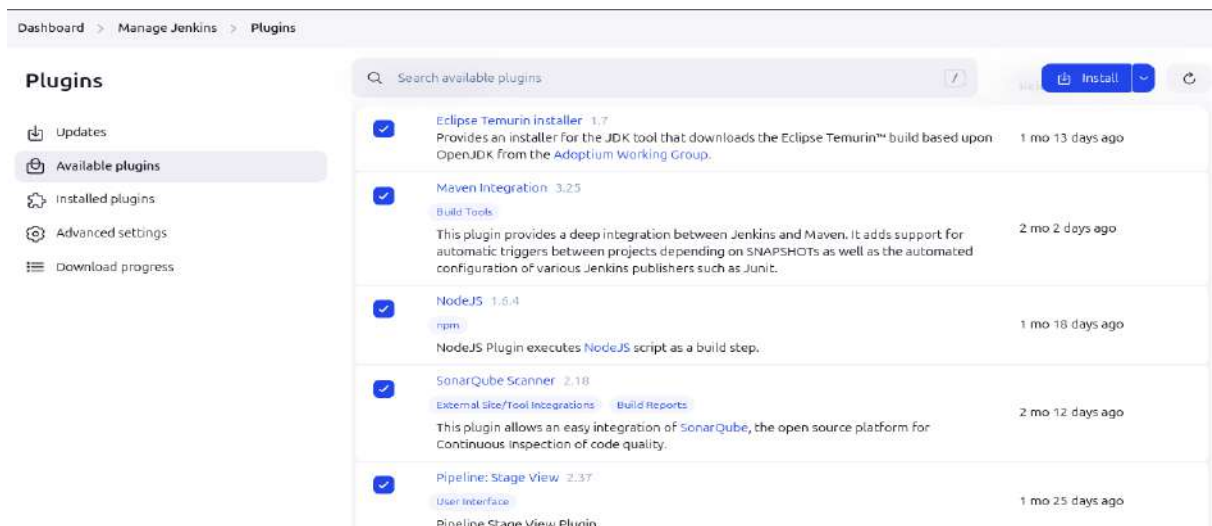


Figure 4.10: Required Plugins for CI pipeline

We configured a number of tools in Jenkins to help our CI/CD pipeline. We added each tool through Manage Jenkins → Global Tool Configuration so that they are available for use in our pipeline jobs.

We selected the Eclipse Temurin JDK 11 to configure the JDK, this is a widely used and stable JDK.



JDK

Name

jdk11

☒ Install automatically ?

Install from adoptium.net ?

Version ?

jdk-11.0.26+4

Add Installer

Figure 4.11: jdk 11

For Maven, we configured version 3. This is one of the most used tools for building Java projects. The feature allows us to create the necessary packages and manage dependencies in our backend services.



Maven

Name

maven3

☒ Install automatically ?

Install from Apache

Version

3.9.9

Figure 4.12: maven 3

Next, we added Node.js and selected Node.js version 12, which was sufficient for our building and testing our front-end application. Node.js support was possible by the NodeJS Plugin we installed earlier.



Figure 4.13: Node 12

Finally, we set version 4.6.0.2311 SonarQube Scanner. This tool let Jenkins send analysis codes to the SonarQube server to monitor bugs and code smells. In short, it helped us improve code quality during CI.



Figure 4.14: Sonar-Scanner

For the Jenkins Setup, we configured credentials to connect it with GitHub, SonarQube... etc. This was important so that Jenkins could fetch source code from the repositories and send the analysis to SonarQube. We managed all the credentials via Jenkins' built-in credentials to store secrets in a secure manner.

First, we added our GitHub credentials In Jenkins, we managed the credentials and added a new credential of type username and password. We entered our GitHub username and personal access token in the password. This allowed Jenkins to login to GitHub and access our private repos during pipeline runs.



The screenshot shows the Jenkins 'Global credentials (unrestricted)' configuration page. The breadcrumb trail at the top is 'Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >'. The 'Kind' is set to 'Username with password'. The 'Scope' is 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' is 'BenSassiAhmed'. There is an unchecked checkbox for 'Treat username as secret'. The 'Password' field is masked with dots. The 'ID' is 'github-credentials'. A blue 'Create' button is at the bottom left.

Figure 4.15: Github Credentials

Then, we generated a token for SonarQube First, we logged into our SonarQube server with the admin user and went to My Account → Security and pressed Generate Token. We gave the token a name and copied it In Jenkins, we included it as a Secret Text credential with global scope. Later on, we will be using this token in our pipeline to authenticate and send the code quality reports to SonarQube.



The screenshot shows the 'Log in to SonarQube' page. It has a title 'Log in to SonarQube'. Below it are two input fields: the first contains 'admin' and the second is masked with dots. At the bottom right are two buttons: 'Log in' and 'Cancel'.

Figure 4.16: Accessing SonarQube

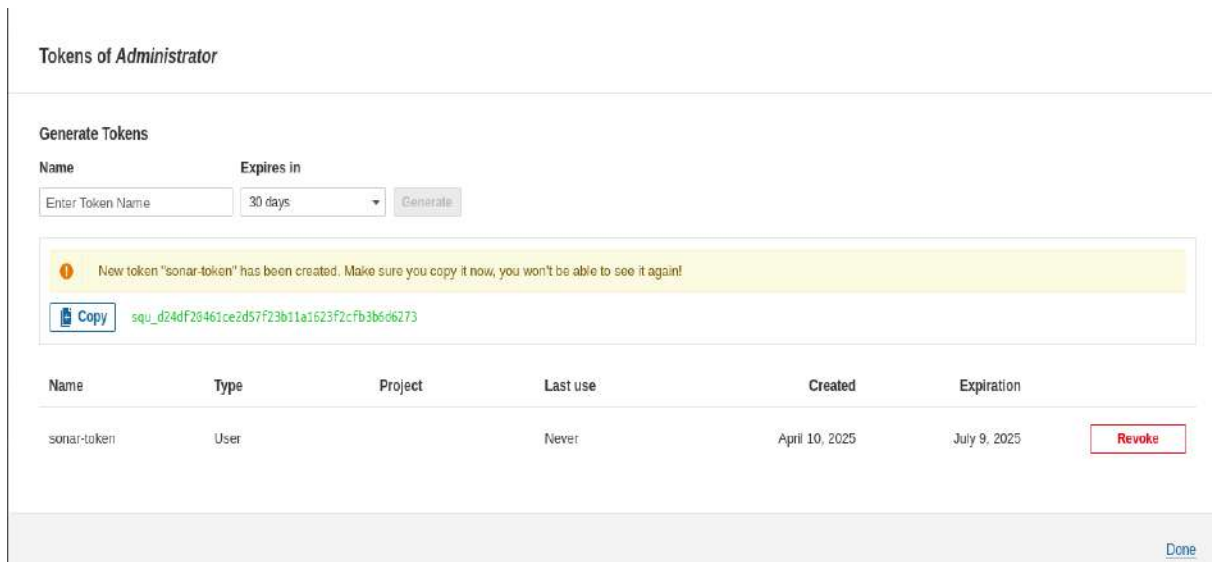


Figure 4.17: Sonar Token



Figure 4.18: Sonar Credentials

After that, we will configure the SonarQube server in Jenkins after storing the token as a secret in Jenkins credentials. In order for Jenkins to connect with our running SonarQube, we needed to configure the SonarQube server in Jenkins to run SonarQube Scanners.

We went to Manage Jenkins → Configure System, and scrolled down to the SonarQube servers section. We defined a name that we could identify with later in the Jenkins pipeline which created a new SonarQube installation there. Under the Server URL, we pasted the URL of our running SonarQube server.

Next, we picked the Server authentication token from credentials drop down which we had added before as a Secret Text. This token helped Jenkins do secure login with SonarQube whenever any scan runs.

Dashboard > Manage Jenkins > System >

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☒ Environment variables

SonarQube installations

List of SonarQube installations:

Name	Server URL	Server authentication token
sonar	http://13.38.78.93:9000	sonar-token

Save Apply

Figure 4.19: Configure SonarQube server in Jenkins

- **Configure Nexus**

After deploying the Nexus instance and verifying that it was running properly, we accessed the Nexus web interface. We signed in through the default details admin user. Then, we created a new hosted repository for the frontend artifacts of our Scrum UI.

We navigated to Repositories and created a new npm (hosted) repository named `scrum-ui-artifacts`. We checked the box that allowed further redeployments. This option was important so we could push multiple builds of the same version for development iterations. We clicked on Create repository leaving rest of the settings as default to complete the process. This repository was later used in our CI pipeline to upload frontend builds onto it through Jenkins using the `npm publish` command.

Sonatype Nexus Repository COMMUNITY 3.79.1-04

Administration

- Repository
 - Repositories**
 - Blob Stores
 - Data Store
 - Proprietary Repositories
 - Content Selectors
 - Cleanup Policies
 - Routing Rules
- Security
 - Privileges
 - Roles

Repositories / Select Recipe / Create Repository: npm (hosted)

Name: A unique identifier for this repository
scrum-ui-artifacts

Online: ☒ If checked, the repository accepts incoming requests

Storage

Blob store:
Blob store used to store repository contents
default

Strict Content Type Validation:
☒ Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format

Hosted

Deployment policy:
Controls if deployments of and updates to artifacts are allowed
Allow redeploy

Figure 4.20: Creating Repository for SCRUM-UI

We also configured the npm Bearer Token Realm for npm clients connecting to our Nexus repository, in order to manage authentication and authorization. We went to Security → Realms, then took the npm Bearer Token Realm from the available realms

and moved it over to the active realms. npm Bearer Token Realm is required for working with token-based npm registries hosted at Nexus.

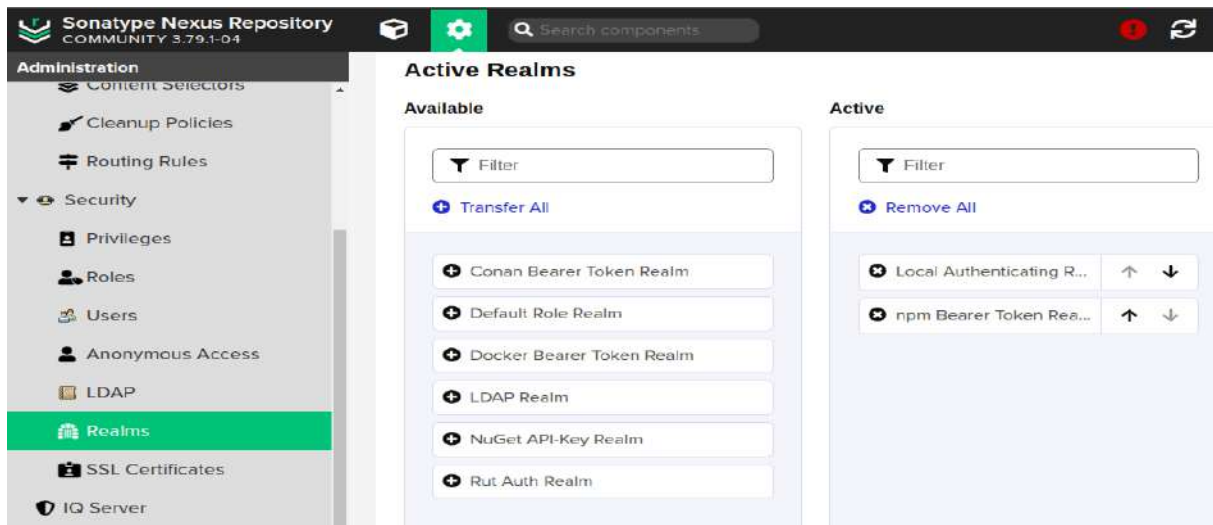


Figure 4.21: Adding Npm Bearer Token Realm

- **Configure Source Code**

To enable SonarQube analysis in our Java backend application, we include the sonar-maven-plugin in the pom.xml file. We included this plugin so that using Jenkins, we could send the code quality metrics to the SonarQube server. We also added the JaCoCo Maven plugin in order to create test coverage reports. SonarQube uses these reports to help us see how well-tested our code is by providing detailed code test coverage metrics.



Figure 4.22: SonarQube and JaCoCo plugins

To configure the pom.xml for Nexus integration, we added the deploy plugin to publish the artifacts. We included the Maven deploy plugin in the pom.xml and also added a distributionManagement section for our Nexus URL. As a result of this, Jenkins was able to push our .jar files to Nexus after a Maven build.

```
<plugin> <!-- define plugin for deploy to nexus -->
|   <groupId>org.apache.maven.plugins</groupId>
|   <artifactId>maven-deploy-plugin</artifactId>
|   <version>3.1.1</version>
| </plugin>

<plugin> <!-- using the plugin to actually deploy to nexus -->
|   <groupId>org.apache.maven.plugins</groupId>
|   <artifactId>maven-deploy-plugin</artifactId>
| </plugin>
```

Figure 4.23: Nexus Plugins

```
<distributionManagement>
|   <snapshotRepository>
|       <id>nexus-snapshots</id>
|       <url>http://15.237.182.199:8081/repository/maven-snapshots/</url>
|   </snapshotRepository>
| </distributionManagement>
```

Figure 4.24: Nexus distribution Management

We changed the package.json file on the front end and added the publishConfig field. The scrum-ui-artifacts repository Nexus URL was provided in this field so that our frontend would be able to publish its build artifacts to Nexus using npm publish.

```
"publishConfig": {
|   "registry": "http://15.237.182.199:8081/repository/scrum-ui-artifacts/"
| },
```

Figure 4.25: Nexus URL in frontend

Conclusion

In sprint 1, we provisioned and configured the continuous integration tools with the help of Terraform which offered us the IaC principle. This established a solid foundation for automating our development process. The next sprint will focus on building and implementing the CI pipeline.

Chapter 5

Sprint 2: Implementing CI pipeline and Docker Integration

Introduction

For Sprint 2, we intended to write the CI pipeline and then to containerize our application and integrate docker in CI pipeline to ensure the application will operate consistently regardless of the environment.

5.1 Sprint 2 Backlog Product

Table 5.1: Sprint 2 Backlog Product

ID	Task	Description
1	Setup CI pipeline	Setup Jenkins CI pipeline and make the necessary configuration.
2	Write CI Pipeline Stages in Jenkinsfile	Define stages for build, test, SonarQube analysis, packaging, and artifact upload to Nexus in the Jenkinsfile.
3	Create Dockerfiles for 'scrum-ui' and 'scrum-app'	Write Dockerfiles to containerize both the 'scrum-ui' (frontend) and 'scrum-app' (backend).
4	Install Docker on Jenkins Server	Install Docker on the Jenkins server to enable building and pushing Docker images.
5	Install and Configure Docker Plugin in Jenkins	Install the Docker plugin and configure Docker tools in Jenkins.
6	Create Docker Repositories on Docker Hub	Create repositories for 'scrum-ui' and 'scrum-app' on Docker Hub.
7	Configure Docker Credentials in Jenkins	Set up Docker Hub credentials in Jenkins for secure access.

ID	Task	Description
8	Integrate Docker into Jenkins Pipeline	Modify Jenkinsfile to build and push Docker images for both ‘scrum-app’ and ‘scrum-ui’.

5.2 Sprint 2 Realisation

- Setting Up CI pipeline

We created a new pipeline in Jenkins to help with the build and deployment of the project. When creating a new item, we first selected type “Pipeline” and named it pfe-2025. We chose to discard a maximum of 5 builds in the configuration so that our workspace remains clean. For writing our pipeline script, we picked to pull the script right from the source control management (SCM), that is GitHub. We provided the right GitHub credentials to give Jenkins access to the repository. Upon checking that everything was OK, we hit OK.

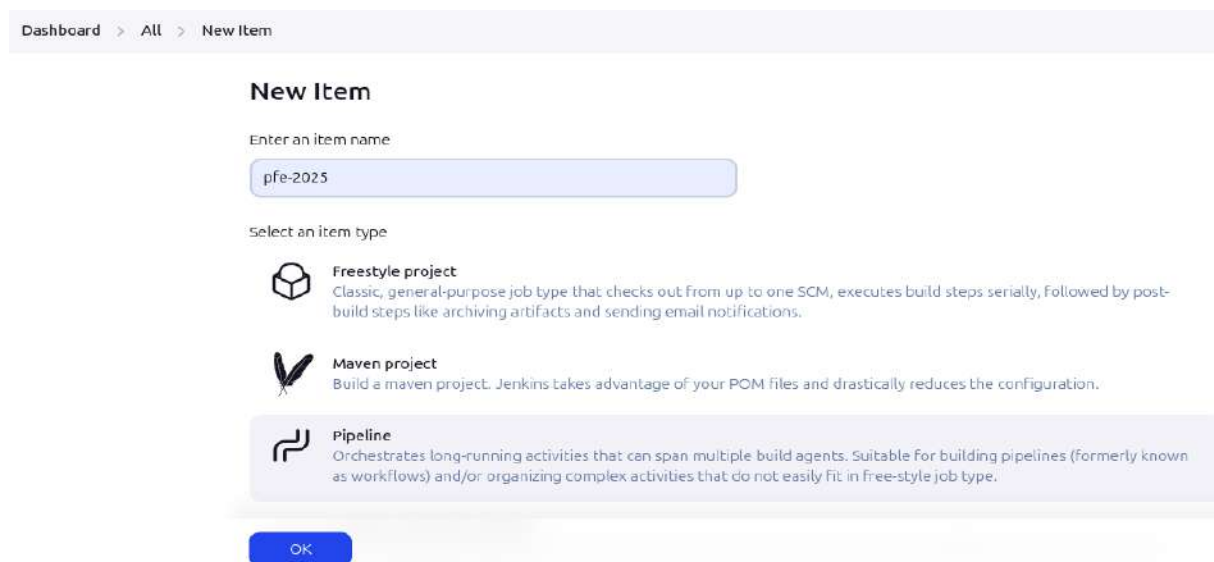


Figure 5.1: Creating Pipeline Type named pfe-2025



The screenshot shows the 'Discard old builds' configuration page in Jenkins. It features a sidebar with a checked box and a question mark icon. The main area is titled 'Strategy' and contains a 'Log Rotation' section. Below this, there are two input fields: 'Days to keep builds' with a hint 'if not empty, build records are only kept up to this number of days' and 'Max # of builds to keep' with a hint 'if not empty, only up to this number of build records are kept'. The 'Max # of builds to keep' field is set to '5'.

Figure 5.2: Max of builds to keep



The screenshot shows the 'Pipeline script from SCM' configuration page in Jenkins. It features a sidebar with a question mark icon. The main area is titled 'Definition' and contains a 'Pipeline script from SCM' section. Below this, there are three input fields: 'SCM' set to 'Git', 'Repositories' (indicated by a dashed box) with a 'Repository URL' set to 'https://github.com/BenSassiAhmed/pfe-2025.git' and 'Credentials' set to 'BenSassiAhmed/*****'.

Figure 5.3: Pipeline Script From SCM

- **Jenkins CI Pipeline**

The CI pipeline automates the build, test, analyse, and deploy of the backend and frontend components of the scrum-app project. The different changes to the code are built, tested, analysed and deployed to n Nexus post verification.

The pipeline begins with a specification declaring a list of tools that are required for the process. These include Maven which deploys the Java backend, JDK 11 and Node12 for the frontend. The environment variables also set the paths for the backend and frontend project folders and the location for the SonarQube scanner tool. The setup here allows the pipeline to have access to this tool's correct versions and executes each step in the stages.

```

1 pipeline {
2     agent any
3
4     tools {
5         maven 'maven3'
6         jdk 'jdk11'
7         nodejs 'node12'
8     }
9
10    environment {
11        SCANNER_HOME=tool 'sonar-scanner'
12        SCRUM_APP_DIR = 'scrum-app'
13        SCRUM_UI_DIR = 'scrum-ui'
14    }
15

```

Figure 5.4: Jenkinsfile Part 1

In the first stage “Compile” we proceed with Compiling both the frontend and backend. For backend, we are using maven to clean and compile the java code. At the same time for the frontend we use npm to download all the dependency required by scrum-ui. This stage ensures that the code is ready for testing

For the second stage "test cases", we run unit tests for the backend and frontend. For the backend, Maven is used to run the tests ,we run Angular tests on the frontend using the ng test command. This stage is to ensure that the code works as expected.

```

16 stages {
17     stage("Compile") {
18         steps {
19             dir("${SCRUM_APP_DIR}") {
20                 sh "mvn clean compile"
21             }
22             dir("${SCRUM_UI_DIR}") {
23                 sh "npm install"
24             }
25         }
26     }
27
28     stage("Test Cases") {
29         steps {
30             dir("${SCRUM_APP_DIR}") {
31                 sh "mvn test jacoco:report"
32             }
33
34             dir("${SCRUM_UI_DIR}") {
35                 sh "npx ng test --code-coverage --watch=false --browsers=ChromeHeadless || true"
36             }
37         }
38     }

```

Figure 5.5: Jenkinsfile Part 2

"Sonarqube Analysis" is the next stage that runs static code analysis using Sonarqube for backend and frontend. In this stage Sonarqube performs the static code analysis with Sonarqube for the back-end as well as front-end.


```

40 stage("Sonarqube Analysis") {
41     steps {
42         dir("${SCRUM_APP_DIR}") {
43             withSonarQubeEnv('sonar') {
44                 sh "' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=scrum-app -Dsonar.projectKey=scrum-app -Dsonar.java.binaries=. '"
45             }
46         }
47     }
48     dir("${SCRUM_UI_DIR}") {
49         withSonarQubeEnv('sonar') {
50             sh "' $SCANNER_HOME/bin/sonar-scanner \
51                 -Dsonar.projectKey=scrum-ui \
52                 -Dsonar.projectName=scrum-ui \
53                 -Dsonar.sources=src \
54                 -Dsonar.exclusions=**/node_modules/**,*/*.spec.ts \
55                 -Dsonar.tests=src \
56                 -Dsonar.test.inclusions=**/*.spec.ts \
57                 -Dsonar.javascript.lcov.reportPaths=coverage/scrum-ui/lcov-info'"
58         }
59     }
60 }
61 }
62 }

```

Figure 5.6: Jenkinsfile Part 3

The “Build” stage compiles the application and packages both backend as well as frontend. To prepare the backend, we will clean and package the Java application using Maven. We use the npm run build command for the frontend that creates build files for the frontend application. This step makes sure both the components are ready to deploy.

The last step is to deploy the built artifacts to Nexus which is called "Push to Nexus". The backend artifact is deployed to Nexus using Maven. The frontend artifact is published to the Nexus repository using npm. This stage will make sure the latest version of the artifacts are available on Nexus.

```

64
65 stage("Build") {
66     steps {
67         dir("${SCRUM_APP_DIR}") {
68             sh "mvn clean package"
69         }
70         dir("${SCRUM_UI_DIR}") {
71             sh "npm run build"
72         }
73     }
74 }
75
76 stage("Push to Nexus") {
77     steps {
78         dir("${SCRUM_APP_DIR}") {
79             sh "mvn deploy -DskipITs"
80         }
81         dir("${SCRUM_UI_DIR}") {
82             sh "npm publish"
83         }
84     }
85 }
86 }
87 }

```

Figure 5.7: Jenkinsfile Part 4

In brief, this CI pipeline automates the full process of compiling testing building and deploying code artifacts. It makes the development process smoother, faster and minimizes the chances of mistakes.

pfe-2025

Add description

Stage View



Figure 5.8: CI pipeline Result

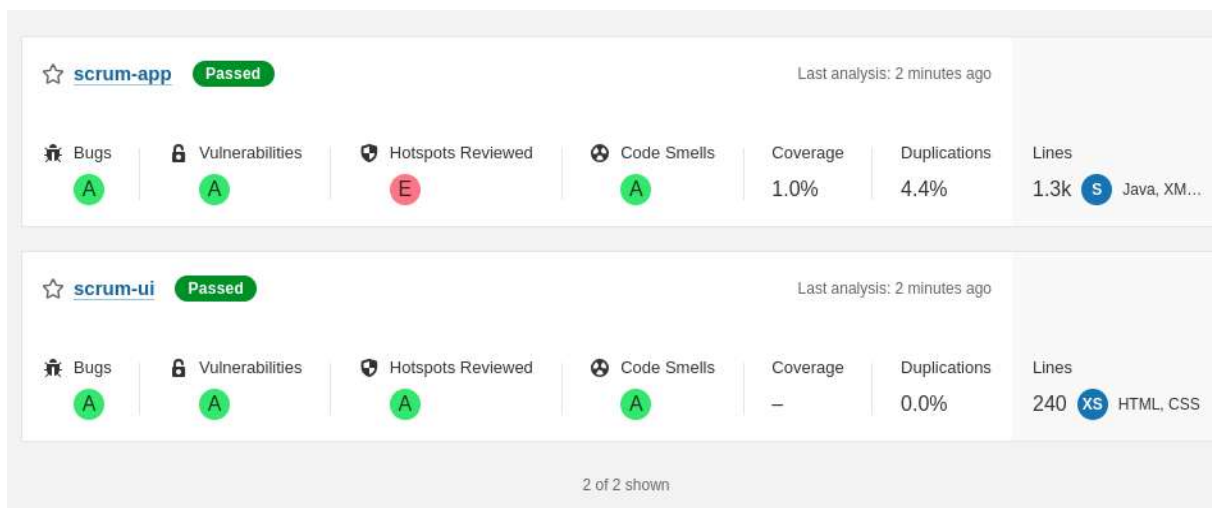


Figure 5.9: SonarQube Results

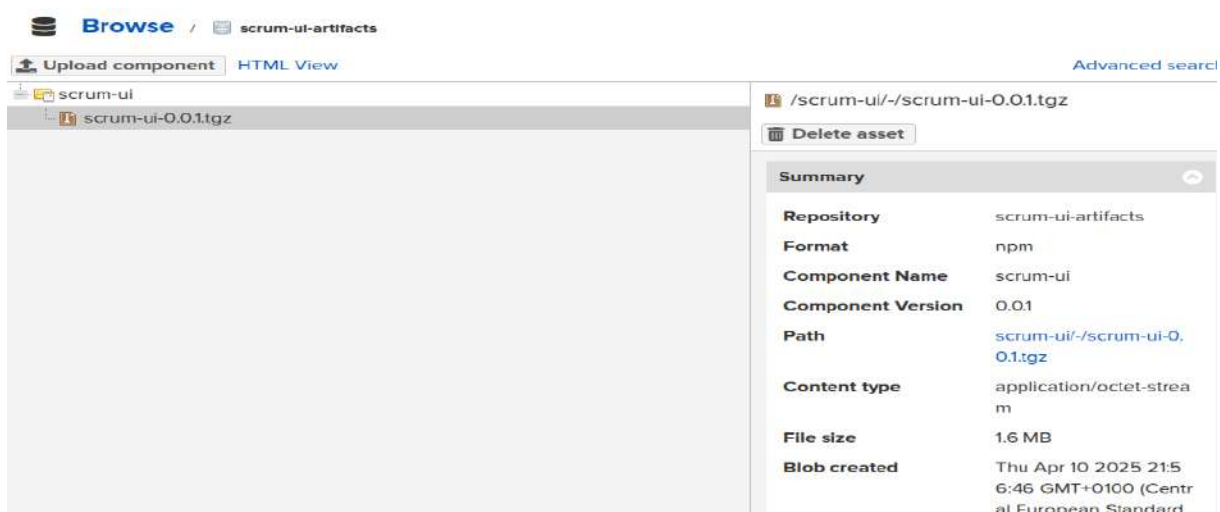


Figure 5.10: SCRUM-UI Artifact

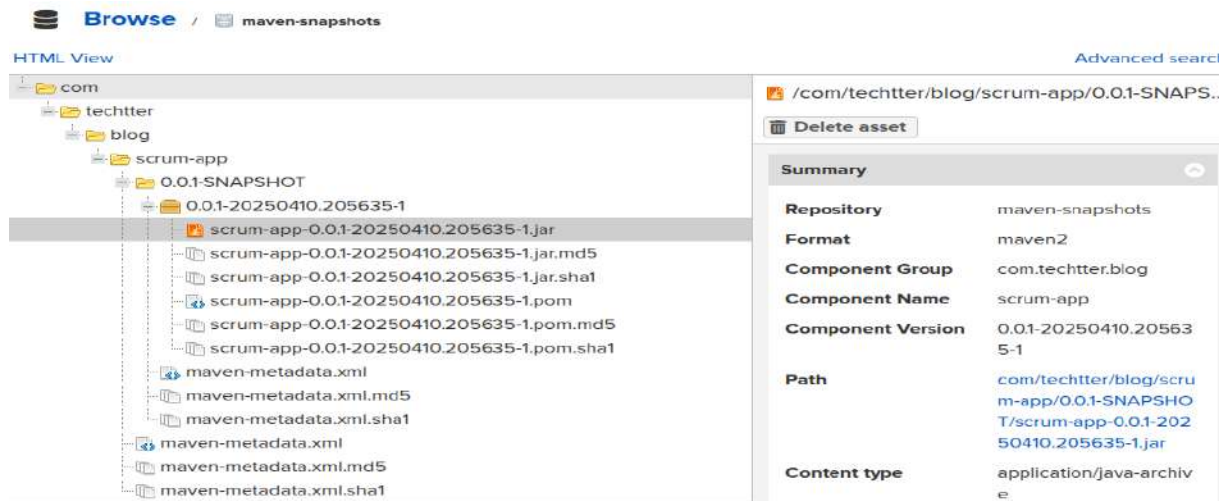


Figure 5.11: SCRUM-APP Artifact

- Create dockerfiles for scrum-ui and scrum-app

Once the CI pipeline was completed successfully, we are moving on to the next important step which is dockerization. By using docker images for backend and frontend apps we eliminated the age-old “it works on my machine” issue. Docker helped us bundle everything up with all necessary dependencies in lightweight containers making it portable and easy to run. And also sets us up well to include container orchestration tools such as Kubernetes in future sprints as well.

We created a multi-stage Dockerfile to containerize our angular frontend application. We made use of node.js alpine image which is light version to build the scrum-ui in the first stage. As a result, we could install all dependencies and build the application. We made this build process separate to the run image process to have a lighter and more efficient docker image. The next stage makes use of the official NGINX image in order to serve the built frontend. We added a custom NGINX config for proper routing setup. This way, the final image size was kept small so the load of the container will be fast.

```

1  ### STAGE 1: Building Stage###
2  FROM node:12.7-alpine AS build
3  WORKDIR /usr/src/app
4  COPY package.json ./
5  RUN npm install
6  COPY . .
7  RUN npm run build
8
9  ### STAGE 2: Run ###
10 FROM nginx:1.17.1-alpine
11 COPY default.conf /etc/nginx/conf.d/default.conf
12 COPY --from=build /usr/src/app/dist/scrum-ui /usr/share/nginx/html
13 EXPOSE 80

```

Figure 5.12: Scrum-ui Dockerfile

We also wrote a multi-stage Dockerfile for our Spring Boot backend application, the scrum-app, which separates the build environment from the execution environment. In

the first step, we use the maven base image to compile the application and create a jar file. This stage ensured resolution of all project dependencies and testing of code before creation of final artifact. We used a minimal Java runtime image in the second stage to run the application. The size of our complete image was reduced and unnecessary build tools were not included in its runtime.

```

1  FROM maven:3.6.1-jdk-8-slim AS build
2  RUN mkdir -p /workspace
3  WORKDIR /workspace
4  COPY pom.xml /workspace
5  COPY src /workspace/src
6  RUN mvn -f pom.xml clean package
7
8  FROM openjdk:8-alpine
9  COPY --from=build /workspace/target/*.jar app.jar
10 EXPOSE 8080
11 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Figure 5.13: Scrum-app Dockerfile

• Configure Docker in Jenkins

we installed docker on Jenkins server. We install it with apt after adding Docker's repo as per the official installation steps for Ubuntu. After installation, we checked that Docker was running with `docker --version`. To manage Docker commands without root privilege, we added jenkins user to docker group.

To install the Docker plugin in Jenkins, we go to "Manage Jenkins" > "Manage Plugins," search for "Docker plugin," and click "Install."



Figure 5.14: Docker plugin

To configure Docker as a tool in Jenkins, we navigated to "Manage Jenkins" > "Global Tool Configuration" and added Docker under the "Docker" section, specifying the Docker executable path. This allowed Jenkins to interact with Docker commands in the pipeline and manage containerized builds.



The screenshot shows the 'Docker' tool configuration page in Jenkins. At the top, there's a section titled 'Docker' with a hamburger menu icon. Below it, the 'Name' field is set to 'docker'. The 'Install automatically' checkbox is checked. Below this, there's a section titled 'Download from docker.com' with a hamburger menu icon. Under this section, the 'Docker version' is set to 'latest'.

Figure 5.15: Docker Tool Configuration

To add docker hub credentials in Jenkins, we first created a personal access token in the docker hub, inside Account Settings > Security > New Access Token . Next, we went to Jenkins, click “Manage Jenkins” > “Manage Credentials” > then add “Username with password” Credentials. We used our Docker Hub username as the user and the generated token as the password. This helped securely authenticate jenkins with docker hub for image push.



The screenshot shows the 'Manage Credentials' page in Jenkins, specifically the 'Global credentials (unrestricted)' section. The 'Kind' is set to 'Username with password'. The 'Scope' is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' is set to 'bensassiahmed'. The 'Treat username as secret' checkbox is unchecked. The 'Password' field is filled with a series of dots. The 'ID' is set to 'dockerhub-credentials'.

Figure 5.16: Docker Tool Configuration

To create Docker Hub repositories for scrum-ui and scrum-app, we logged in to Docker Hub and created two repositories. We went to the 'Repositories' section on Docker Hub and clicked on Create Repository. For each repository we mentioned names as "scrum-ui" and "scrum-app" respectively as these names should be unique to your account. We use these repositories as a destination to store the Docker images built and pushed from Jenkins.

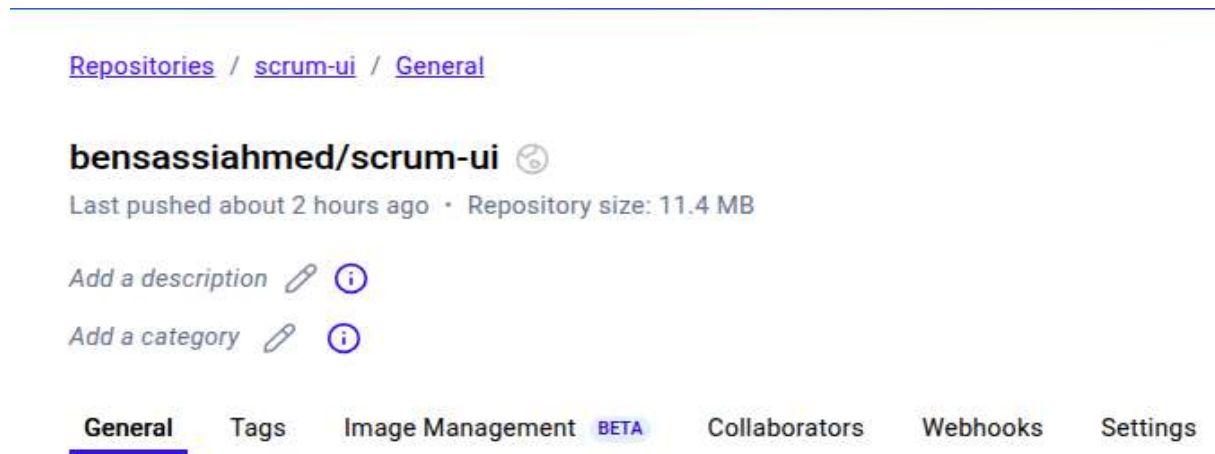


Figure 5.17: Scrum-ui dockerhub repository

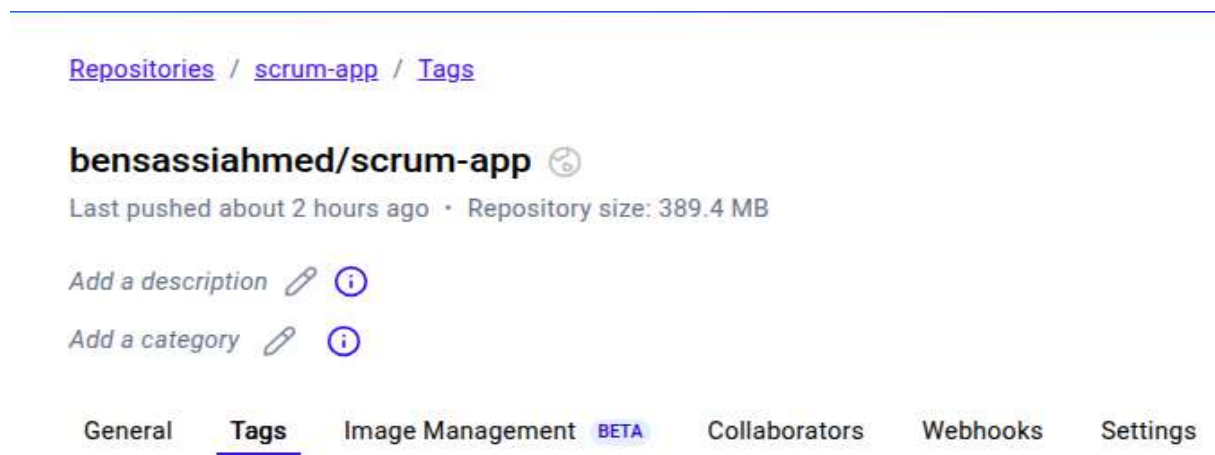


Figure 5.18: Scrum-app dockerhub repository

These Docker Environment Variables were added for Jenkins as mentioned below. They let us specify our docker hub repositories for the scrum-app and scrum-ui. Also we can securely manage our docker credentials. We benefited from the Jenkins build number to tag our images so that every image pushed to Docker Hub had a version. By using this setup, we could manage image names, tags, and credentials thus making it clean and consistent throughout pipeline stages.

```
environment {  
    SCANNER_HOME=tool 'sonar-scanner'  
    SCRUM_APP_DIR = 'scrum-app'  
    SCRUM_UI_DIR = 'scrum-ui'  
    DOCKER_CREDENTIALS_ID = 'dockerhub-credentials'  
    SCRUM_APP_REPO      = 'bensassiahmed/scrum-app'  
    SCRUM_UI_REPO       = 'bensassiahmed/scrum-ui'  
    SCRUM_APP_IMAGE     = "${SCRUM_APP_REPO}:${BUILD_NUMBER}"  
    SCRUM_UI_IMAGE      = "${SCRUM_UI_REPO}:${BUILD_NUMBER}"  
}
```

Figure 5.19: used environment variables for docker integration

At this stage, we created Docker images for scrum-app and scrum-ui projects. We did use docker build command with the image tags we defined earlier which included Jenkins build number so that we can version it. We had to package our source code and dependencies into images which are portable to be able to push to DockerHub.

```
stage("Build Images") {  
    steps {  
        sh """  
            docker build -t ${SCRUM_APP_IMAGE} ${SCRUM_APP_DIR}  
            docker build -t ${SCRUM_UI_IMAGE} ${SCRUM_UI_DIR}  
        """  
    }  
}
```

Figure 5.20:

During the "Docker Login" stage, we authenticated Jenkins with Docker Hub using stored credentials. Through Jenkins credentials, our Docker Hub username and access token was securely retrieved to allow the pipeline to log in without revealing secrets. We needed to perform this step to let Jenkins push the Docker images we built in the next step to the Docker Hub repositories.


```

stage("Docker Login") {
  steps {
    withCredentials([usernamePassword(credentialsId: "${DOCKER_CREDENTIALS_ID}",
    usernameVariable: 'DOCKER_USER',
    passwordVariable: 'DOCKER_PASS')]) {
      sh '''
        echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
      '''
    }
  }
}

```

Figure 5.21: Docker Login Stage

At this stage called “Docker Push Images”, we pushed the Scrum app and Scrum UI Docker images created on the local system to the Docker Hub. This step made sure our built Docker images of Scrum app and Scrum UI are pushed to docker hub repos to easily access from different environments or other team members.

```

stage("Docker Push Images") {
  steps {
    sh '''
      docker push ${SCRUM_APP_IMAGE}
      docker push ${SCRUM_UI_IMAGE}
    '''
  }
}

```

Figure 5.22: Docker Push Images Stage

- Result of improved CI pipeline

After configuring and integrating docker to our CI pipeline we executed the job and we could see that docker images are available in Dockerhub .



Figure 5.23: Scrum-ui docker image

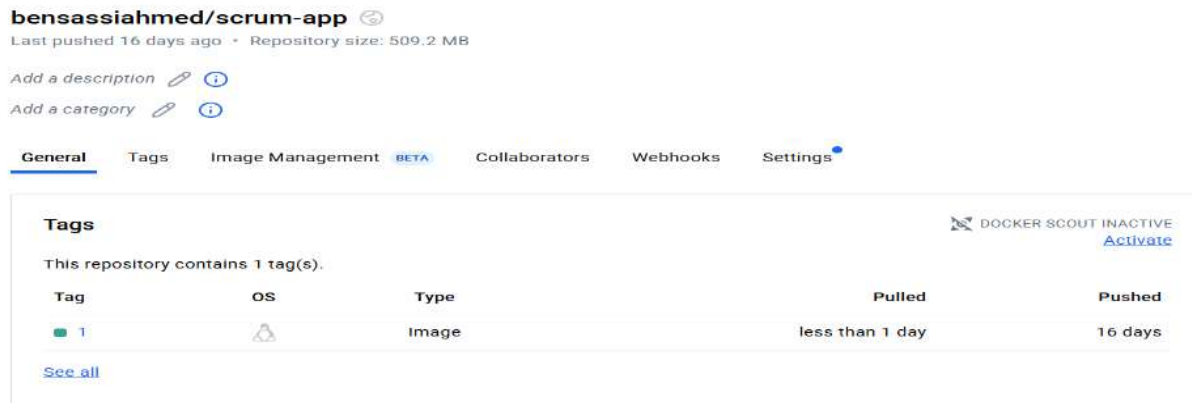


Figure 5.24: Scrum-app docker image

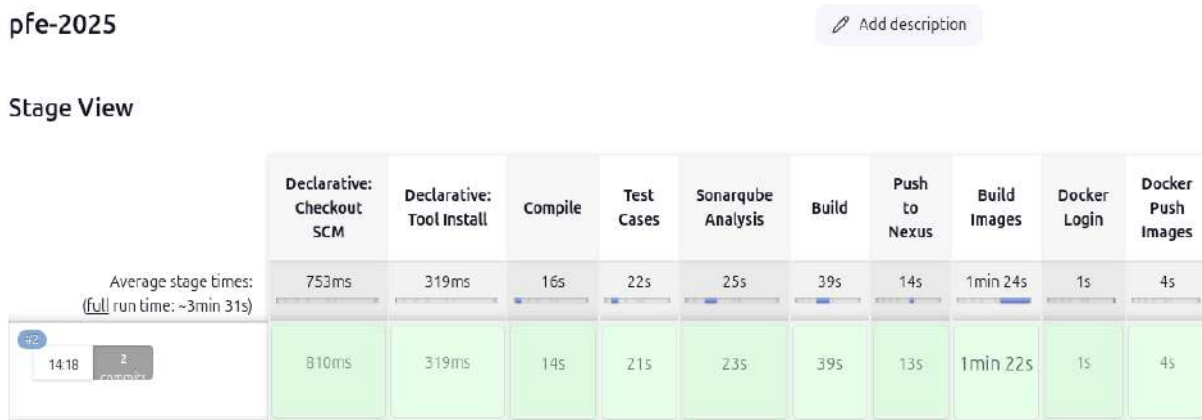


Figure 5.25: Improved CI pipeline result

Conclusion

During this sprint, we set up the Continuous Integration (CI) pipeline from code build and testing to SonarQube for code quality analysis and Nexus for artifact management. We improved the pipeline even more by including Docker which helped us build our frontend and backend in docker images and then push them to Docker Hub. This made our pipeline more reliable and our application ready for deployment in any environment. In the future sprint, we will set up a Kubernetes cluster and make a CD pipeline for easy and effective deployment of our containerized applications and enabling container orchestration.

Chapter 6

Sprint 3: Designing a Highly Available and Auto-Scaling Kubernetes Cluster

Introduction

During Sprint 3, we worked on deploying Kubernetes for managing and running our app. After creating our CI pipeline and containers in the previous sprints, this sprint is to ensure the system can run in the cloud. We made a set-up that can recover from problems and can automatically adjust to more users whenever required. This sprint helped prepare our project for real-world use.

6.1 Sprint 3 Backlog Product

Table 6.1: Sprint 3 Backlog Product

ID	Task	Description
1	Update VPC and Subnets	In Sprint 1 we created only public subnets, within this Sprint we will add 2 private subnets across different AZs to host our K8s cluster.
2	IAM Configuration	Create and attach IAM roles and policies for the Cluster Autoscaler and AWS Load Balancer Controller to access necessary AWS services.
3	Provision Bastion Node	Deploy a bastion host in the public subnet to enable secure SSH access to private EC2 instances.
4	Provision Master Node	Launch a master node (control plane) in the private subnet to act as the central component of the Kubernetes cluster.
5	Provision Launch Template and ASG	Create an AWS launch template and provision an Auto Scaling Group to manage Kubernetes worker nodes dynamically.

ID	Task	Description
6	Automate Worker Node Patching	Write and schedule a shell script using crontab to automatically patch worker nodes with required labels for the AWS controller.
7	K8s Manifest Files	Prepare and create the K8s manifest files of our application.

6.2 Sprint 3 Realisation

• Updating VPC and Subnets

During sprint 3, our configuration of VPC was extended for deploying Kubernetes cluster in a more secure and production-like environment. In Sprint 1 we created only public subnets, within this Sprint we will add 2 private subnets across different availability zones to host our Kubernetes master and also our worker nodes.

We created a NAT Gateway to allow these private instances connectivity by downloading packages or communicating to AWS services over the internet. We continued to utilize the official Terraform AWS VPC module, which streamlined the provisioning process and ensured reusability.

We kept using `data.aws-availability-zones.available` to allow zones to fetch dynamically and we kept using `cidrsubnet()` to calculate our subnets from the VPC main range. This made our network setup more flexible and scalable. We consistently applied tags using variables to maintain a clean and efficient infrastructure within AWS.

```
data "aws_availability_zones" "available" {
  state = "available"
}

You, 4 days ago | 1 author (You)
module "demo-k8s-aws-vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.17.0"

  name = var.vpc_name
  cidr = var.cidr_block

  azs = [data.aws_availability_zones.available.names[0], data.aws_availability_zones.available.names[1]]
  public_subnets = [cidrsubnet(var.cidr_block, 8, 10), cidrsubnet(var.cidr_block, 8, 20)]
  private_subnets = [cidrsubnet(var.cidr_block, 8, 110), cidrsubnet(var.cidr_block, 8, 120)]

  create_igw = true
  enable_dns_hostnames = true

  enable_nat_gateway = true
  create_private_nat_gateway_route = true

  tags = var.tags
  public_subnet_tags = var.public_subnet_tags
}
```

Figure 6.1: Updating VPC and Subnets

• IAM Configuration

We created an IAM role and attached policies to it for the AWS Load Balancer Controller and the Cluster Autoscaler. These are necessary to give these components the right per-

missions to interact with AWS resources.

The IAM role (`demo-k8s-aws-combined-role`) is created with a policy that allows EC2 instances (the Kubernetes worker nodes) to assume the role. This allows the worker nodes to access AWS services like load balancers and scaling features.

We attached two policies to this role:

- **ALB Controller Policy:** This gives the ALB Controller permissions to manage load balancers and route traffic to Kubernetes pods.
- **Cluster Autoscaler Policy:** This allows the Cluster Autoscaler to scale worker nodes up or down based on demand, ensuring that the Kubernetes cluster has enough resources.

An IAM instance profile (`demo-k8s-aws-combined-instance-profile`) is created and associated with the role. This profile is then attached to the EC2 worker nodes, enabling them to use the permissions defined in the IAM role.

These IAM resources are crucial for ensuring that the ALB Controller and Cluster Autoscaler can function properly. Without them, the services wouldn't be able to manage load balancing or scale the worker nodes as needed.

The IAM policies for both components are detailed in the annex.

```
# IAM Role for both ALB Controller and Cluster Autoscaler
You, 6 days ago | 1 author (You)
resource "aws_iam_role" "demo-k8s-aws-combined-role" {
  name = "demo-k8s-aws-combined-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect = "Allow",
      Principal = {
        Service = ["ec2.amazonaws.com"]
      },
      Action = "sts:AssumeRole"
    }]
  })
}

# IAM Policy Attachment for ALB Controller permissions
You, 6 days ago | 1 author (You)
resource "aws_iam_role_policy_attachment" "aws-lb-policy" {
  policy_arn = aws_iam_policy.demo-k8s-aws-alb-custom-policy.arn
  role       = aws_iam_role.demo-k8s-aws-combined-role.name
}

# IAM Policy Attachment for Cluster Autoscaler permissions
You, 6 days ago | 1 author (You)
resource "aws_iam_role_policy_attachment" "cluster-autoscaler-policy-attachment" {
  policy_arn = aws_iam_policy.cluster-autoscaler-policy.arn
  role       = aws_iam_role.demo-k8s-aws-combined-role.name
}

# Instance Profile for the Worker Node
You, 6 days ago | 1 author (You)
resource "aws_iam_instance_profile" "demo-k8s-aws-combined-instance-profile" {
  name = "demo-k8s-aws-combined-instance-profile"
  role = aws_iam_role.demo-k8s-aws-combined-role.name
}
```

Figure 6.2: IAM for ALB Controller and Cluster Autoscaler

- **Provision Bastion Instance**

The purpose of the bastion host is to provide secure SSH access to our private Kubernetes cluster. It acts as a gateway, allowing us to connect to internal resources without exposing

them directly to the internet. To enhance security, we configured SSH access to the Bastion only to our machine's IP address.

```
#Bastion Host
You, 2 weeks ago | 1 author (You)
resource "aws_instance" "demo-bastion-node" {
  ami            = data.aws_ami.latest_ami.id
  instance_type  = var.bastion_instance_type
  key_name       = var.key_pair
  associate_public_ip_address = true
  vpc_security_group_ids = [
    aws_security_group.demo-bastion-sg.id
  ]
  subnet_id = module.demo-k8s-aws-vpc.public_subnets[0]
  You, 2 months ago | 1 author (You)
  root_block_device {
    volume_size = var.volume_size
  }
  tags = merge(var.tags, {
    Name = var.bastion_host_name
  })
}
```

Figure 6.3: Bastion EC2 instance

- **Provision Control Plane**

We used Terraform to provision the Kubernetes master node in AWS. Following security best practices, the master node was deployed in a private subnet without a public IP. To access and manage the cluster, a bastion host was deployed in a public subnet. We connected to the bastion host to run kubectl commands and interact with the Kubernetes API securely.

A particular security group was utilized to restrict only the necessary traffic, and an SSH key pair was included to connect to the instance through the bastion host. We also set the root volume size and created a name tag for the instance to identify it easily.

At this stage, the EC2 instance was a base setup. We connected to it and installed the necessary components to initialize it as the Kubernetes master node. We will also install the HPA, VPA, and CA .

```
resource "aws_instance" "demo-k8s-master-node" {
  ami            = "ami-0f462ea4c2f9c4dd7"
  instance_type  = var.cluster_node_type
  iam_instance_profile = aws_iam_instance_profile.demo-k8s-aws-combined-instance-profile.name
  key_name       = var.key_pair
  You, last month • solved conflicts •
  vpc_security_group_ids = [
    aws_security_group.k8s_control_plane_sg.id
  ]
  subnet_id = module.demo-k8s-aws-vpc.private_subnets[0]
  You, last month | 1 author (You)
  root_block_device {
    volume_size = var.volume_size
  }
  tags = merge(var.tags, {
    Name = var.master_node_name
  })
}
```

Figure 6.4: Control Plane Provisionning

- **Aws Launch Template**

Since our master node has been successfully set up and we have extracted its `kubeadm join` command, it is time to provision the worker nodes and attach them to the cluster.

An AWS Launch Template will define how the worker nodes must be created. The worker node will have an AMI. This AMI has been preconfigured using Packer to install all the tools needed for a worker node in Kubernetes. We specified instance type, key pair, IAM role, and attached a security group that allow communication with the control plane.

In the launch template's user data section, we added a bit of code to allow each worker node to automatically join our Kubernetes cluster using the `kubeadm join` command we had earlier received. This made it so easy that I didn't have to SSH into each instance.

We were able to tag both the instance and its root volume. By having this setup in place, the worker nodes were scaled later on using the ASG.

```
resource "aws_launch_template" "demo-k8s-aws-worker-lt" {
  name_prefix    = "demo-k8s-aws-worker-"
  image_id       = "ami-0f462ea4c2f9c4dd7"
  instance_type  = var.cluster_node_type
  key_name       = var.key_pair

  You, 6 days ago | 1 author (You)
  iam_instance_profile {
    name = aws_iam_instance_profile.demo-k8s-aws-combined-instance-profile.name
  }

  You, 4 days ago | 1 author (You)
  network_interfaces {
    #associate_public_ip_address = false
    security_groups = [
      aws_security_group.k8s_worker_sg.id
    ]
  }

  You, 6 days ago | 1 author (You)
  block_device_mappings {
    device_name = "/dev/sda1"
    You, 6 days ago | 1 author (You)
    ebs {
      volume_size = var.volume_size
      volume_type = "gp2"
      delete_on_termination = true
    }
  }
}
```

Figure 6.5: Aws Launch Template part 1

```
user_data = base64encode<<-EOF
#!/bin/bash

# Kubernetes join command
kubeadm join 172.32.10.31:6443 --token f7jrog.ng8ymc44yo7h2z39 --discovery
EOF

# Tag specification for EC2 instance and volume
You, 5 minutes ago | 1 author (You)
tag_specifications {
  resource_type = "instance"
  tags = merge(var.tags, {
    Name = "worker-${terraform.workspace}-${timestamp()}" #instance name
  })
}

You, 4 days ago | 1 author (You)
tag_specifications {
  resource_type = "volume"
  tags = merge(var.tags, {
    Name = "worker-volume-${terraform.workspace}-${timestamp()}"
  })
}
```

Figure 6.6: Aws Launch Template part 2

- **Auto Scaling Group**

After the launch template was created, we created the ASG to manage the Kubernetes worker nodes. This makes our cluster highly scalable and fault-tolerant. The auto-scaling group has been configured to have minimum instances set to 2, maximum instances set to 4 and desired capacity set to 2. This ensures that there are always EC2 instances running as workers. These instances in two private subnets in different availability zones for fault tolerance and high availability.

The ASG will use the launch template we created above. That includes the AMI (baked with Kubernetes components using Packer), instance profile (IAM permissions), security groups, EBS configuration, and most importantly a kubeadm join command in the user-data script. Each instance created by the ASG joins the Kubernetes cluster (as a worker node) automatically on launch.

In the Auto Scaling Group we add certain tags to enable Cluster Autoscaler support:

- `k8s.io/cluster-autoscaler/enabled = true` : tells the Cluster Autoscaler that this ASG should be managed.
- `k8s.io/cluster-autoscaler/kubernetes = owned` : tells CA this ASG belongs to the current k8s cluster named `kubernetes`.

Workflow and Trigger Mechanism

When the Kubernetes cluster doesn't have enough resources to run new pods, the CA running inside the cluster notices this. It checks the ASG that is tagged for autoscaling.

If more nodes are needed, the CA uses its IAM permissions to ask the ASG to create a new EC2 instance.

The ASG launches a new instance using the Launch Template we defined. This instance runs a startup script with the kubeadm join command, which makes it automatically join the cluster as a worker node.

This process allows the cluster to scale automatically, handle more workloads when needed, and save costs when fewer nodes are required without any manual steps.


```
# Auto Scaling Group for worker nodes
You, 6 minutes ago | 1 author (You)
resource "aws_autoscaling_group" "workers" {
  name                = "kubeadm-workers-asg"
  min_size            = 2
  max_size            = 4
  desired_capacity    = 2
  vpc_zone_identifier = [module.demo-k8s-aws-vpc.private_subnets[0],
                        module.demo-k8s-aws-vpc.private_subnets[1]]

  You, 6 days ago | 1 author (You)
  launch_template {
    id      = aws_launch_template.demo-k8s-aws-worker-lt.id
    version = "$Latest"
  }

  # Tags for Auto Scaling Group
  You, last week | 1 author (You)
  tag {
    key          = "k8s.io/cluster-autoscaler/enabled"
    value        = "true"
    propagate_at_launch = true
  }

  You, 6 days ago | 1 author (You)
  tag {
    key          = "k8s.io/cluster-autoscaler/kubernetes"
    value        = "owned"
    propagate_at_launch = true
  }
}
```

Figure 6.7: Worker Nodes Auto Scaling Group

- **Patch Worker Nodes**

The script below patches the worker nodes so that they will have the correct provider ID that contains the AWS zone availability and instance ID.

The providerID field is used by the AWS Load Balancer Controller to correctly identify the EC2 instances (nodes). If this field is not present or incorrect, the controller will not attach the successfully the targets to the load balancer.

This script allow the AWS Load Balancer Controller to identify the worker nodes and interact with them.

In order for this to happen automatically, we will run this script in master node using crontab for every 5 minutes so that the addition of worker nodes in the cluster can happen seamlessly without user intervention.

```
#!/bin/bash
set -euo pipefail
export PATH=$PATH:/usr/bin:/usr/local/bin
export REGION="eu-west-3"
export KUBECONFIG="/home/ubuntu/.kube/config"

echo "🔧 Patching worker nodes with AZ-based providerID..."
worker_nodes=$(/usr/bin/kubectl get nodes \
  --selector='!node-role.kubernetes.io/master,!node-role.kubernetes.io/control-plane' \
  -o jsonpath='{.items[*].metadata.name}')

for node in $worker_nodes; do
  current_providerID=$(/usr/bin/kubectl get node "$node" -o jsonpath='{.spec.providerID}')
  if [[ -z "$current_providerID" || "$current_providerID" == aws:/// ]]; then
    echo "⚠️ Node $node has providerID: '$current_providerID'. Updating to AZ format..."
    node_ip=$(/usr/bin/kubectl get node "$node" -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}')
    instance_id=$(/usr/local/bin/aws ec2 describe-instances \
      --region "$REGION" \
      --filters "Name=private-ip-address,Values=$node_ip" \
      --query 'Reservations[*].Instances[*].InstanceId' \
      --output text)

    az=$(/usr/local/bin/aws ec2 describe-instances \
      --region "$REGION" \
      --instance-ids "$instance_id" \
      --query 'Reservations[*].Instances[*].Placement.AvailabilityZone' \
      --output text)

    new_providerID="aws:///az/$instance_id"
    patch="{\"spec\":{\"providerID\":\"$new_providerID\"}}"

    /usr/bin/kubectl patch node "$node" -p "$patch"
    echo "✅ Patched $node with providerID $new_providerID"
  else
    echo "✔️ $node already has proper AZ-based providerID: $current_providerID"
  fi
done
```

Figure 6.8: patch worker nodes script

- **Ingress Configuration**

We created this Ingress file to send traffic from AWS ALB to the scrum-ui service.

When performing the annotations, we configured the ALB with HTTP port 80 and set the load balancer name scrum-ui-alb with internet-facing. We set the target type to instance and added the necessary tags.

The rule we defined routes traffic from the / path to the scrum-ui service on port 80.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: scrum-ui-ingress
  annotations:
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
    alb.ingress.kubernetes.io/load-balancer-name: scrum-ui-alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: instance
    alb.ingress.kubernetes.io/tags: app=scrum, devops-demo=true
spec:
  ingressClassName: alb
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: scrum-ui
            port:
              number: 80
```

Figure 6.9: ingress yaml file

- **Scrum-App K8s files**

In our Kubernetes deployment file of scrum-app, we configured a ConfigMap and a Secret. ConfigMap stores non-sensitive data like DB-SERVER, POSTGRES-DB and other environment variables. We included these in the deployment using the valueFrom field in the environment part in the container.

We used a Secret for sensitive information, such as POSTGRES-USER and POSTGRES-PASSWORD. It ensures sensitive data is made secure within the cluster. These values are also inserted into the deployment's environment variables using secretKeyRef.

We also defined requests and limits for resources so the pod does not consume excessive resources.

We also created a HPA for the scrum-app deployment to be scaled automatically based on CPU. The HPA sets the pod's count between 1 and 10.

The application is exposed internally by the scrum-app service on port 8080

To sum up, we have created ConfigMap and Secret for the application, implemented autoscaling, and created the service for the application.

```
apiVersion: v1
kind: Service
metadata:
  name: scrum-app
spec:
  selector:
    app: scrum-app
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
```

Figure 6.10: scrum-app service yaml file


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scrum-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: scrum-app
  template:
    metadata:
      labels:
        app: scrum-app
    spec:
      containers:
        - name: scrum-app
          image: bensassiahmed/scrum-app:1
          ports:
            - containerPort: 8080
          env:
            - name: DB_SERVER
              valueFrom:
                configMapKeyRef:
                  name: scrum-app-config
                  key: DB_SERVER
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: scrum-app-config
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: scrum-app-secret
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: scrum-app-secret
                  key: POSTGRES_PASSWORD
          resources:
            requests:
              cpu: "500m"
              memory: "512Mi"
            limits:
              cpu: "1000m"
              memory: "1024Mi"
```

Figure 6.11: scrum-app deployment yaml file

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: scrum-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: scrum-app
  minReplicas: 1
  maxReplicas: 3
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Figure 6.12: scrum-app hpa yaml file

- **Scrum-Postgres K8s files**

We set up the application database in the scrum-postgres deployment. To inject the database name, we made use of a ConfigMap, for user name and password we used a Secret. The container received these values using environment variable flags.

We manage memory and CPU usage with appropriate resource requests and limits. We used a Vertical Pod Autoscaler (VPA) to optimize the allocation of resources. The pod's CPU and memory will be adjusted automatically as per usage but within the defined min and max limits.

In the end, we exposed our database using a ClusterIP service on port 5432 for internal access from other components.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scrum-postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: scrum-postgres
  template:
    metadata:
      labels:
        app: scrum-postgres
    spec:
      containers:
        - name: scrum-postgres
          image: postgres:9.6-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: scrum-postgres-config
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: scrum-postgres-secret
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: scrum-postgres-secret
                  key: POSTGRES_PASSWORD
          resources:
            requests:
              cpu: "250m"
              memory: "512Mi"
            limits:
              cpu: "1"
              memory: "1Gi"
```

Figure 6.13: scrum-postgres deployment yaml file

```
apiVersion: v1
kind: Service
metadata:
  name: scrum-postgres
spec:
  selector:
    app: scrum-postgres
  ports:
    - protocol: TCP
      port: 5432
      targetPort: 5432
```

Figure 6.14: scrum-postgres service yaml file

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: scrum-postgres-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: scrum-postgres
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
      - containerName: scrum-postgres
        minAllowed:
          cpu: "100m"
          memory: "256Mi"
        maxAllowed:
          cpu: "1"
          memory: "1Gi"
```

Figure 6.15: scrum-postgres vpa yaml file

- Scrum-UI K8s files

We created the scrum-ui deployment file and we set the docker image that we built in the sprint 2 , we also set requests and limits for CPU and memory so the pod can use resources effectively. There were no environment variables nor secrets needed for this component.

To enable scaling, we wrote a horizontal pod autoscaler file and configured it to scale up and down between pod replicas from 1 to 10 based on cpu usage. We created a NodePort service type in the scrum-ui service file that listens to port 30000 and forwards the traffic to the pod. The AWS Load Balancer Controller uses the NodePort service to direct external traffic to the application.

The scrum-ui would not be overloaded because it could scale when needed; it would run efficiently and remain reachable through the load balancer.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scrum-ui
spec:
  replicas: 1
  selector:
    matchLabels:
      app: scrum-ui
  template:
    metadata:
      labels:
        app: scrum-ui
    spec:
      containers:
        - name: scrum-ui
          image: bensassiahmed/scrum-ui:1
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
```

Figure 6.16: scrum-ui deployment yaml file

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: scrum-ui-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: scrum-ui
  minReplicas: 1
  maxReplicas: 3
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Figure 6.17: scrum-ui hpa yaml file

```
apiVersion: v1
kind: Service
metadata:
  name: scrum-ui
spec:
  selector:
    app: scrum-ui
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30000
  type: NodePort
```

Figure 6.18: scrum-ui service yaml file

Conclusion

In Sprint 3 we created a Kubernetes Cluster capable of handling more traffic and recovering from issues. Our app can now be run in real conditions thanks to this strong base. In the next sprint, we will deploy the application to the k8s cluster and we will try out the autoscaling thing and see how it works.

Chapter 7

Sprint 4: From Deployment to Insight: GitOps with ArgoCD and Real-Time Observability

Introduction

In our last sprint, we focused on showing the GitOps workflow using ArgoCD when something is pushed on git. We tried out the K8s autoscaling features. Ultimately, we took advantage of Prometheus and Grafana for real time observability of the system for performance insights.

7.1 Final Sprint Product Backlog

Table 7.1: Sprint 4 Product Backlog

ID	Task	Description
1	Prepate GitOps Repository	Create GitOps repository containning all the K8s manifest files of our project.
2	Update Docker Version in GitOps Repository	Keeping the GitOps up to date with new docker image versions.
3	Email Notification	The email notification will inform us with the CI pipeline status.
4	GitOps Setup with ArgoCD	Install and configure ArgoCD to be able to sync our created GitOps repository and deploy our application.
5	Autoscaling Demo	Simulate load on the application and observe how HPA and CA respond by scaling the pods and nodes.

ID	Task	Description
6	Grafana Dashboard	Access the Grafana dashboard to view the real-time cluster information, including performance and resource utilization.

7.2 Final Sprint Realisation

We organized our GitOps architecture in a dedicated Git repository that contained all the Kubernetes manifest files. These files defined the deployments, services, secrets, ConfigMaps, autoscaling policies, and Ingress rules, allowing us to manage and deploy the entire application in a consistent and automated manner using ArgoCD.

```
k8s manifest files/  
├── ingress.yaml  
├── scrum-app-configmap.yaml  
├── scrum-app-deployment.yaml  
├── scrum-app-hpa.yaml  
├── scrum-app-secret.yaml  
├── scrum-app-service.yaml  
├── scrum-postgres-configmap.yaml  
├── scrum-postgres-deployment.yaml  
├── scrum-postgres-secret.yaml  
├── scrum-postgres-service.yaml  
├── scrum-postgres-vpa.yaml  
├── scrum-ui-deployment.yaml  
├── scrum-ui-hpa.yaml  
└── scrum-ui-service.yaml
```

Figure 7.1: GitOps Repository Structure

In this stage of the pipeline in Jenkins, we automated the process of updating the GitOps repository with the new docker image versions. Each time a new build was created, the corresponding image tags in the Kubernetes manifest files were updated, committed, and pushed to the repository. This ensured that ArgoCD could detect the changes and trigger a deployment with the latest application versions.

```

stage("Update K8s Manifests in GitOps Repo") {
  steps {
    script {
      withCredentials([usernamePassword(
        credentialsId: env.GITHUB_CREDS,
        passwordVariable: 'GIT_PASSWORD',
        usernameVariable: 'GIT_USERNAME'
      )]) {
        // Clone GitOps repo
        sh 'rm -rf gitops-pfe-2025 || true'
        sh "git clone https://$${GIT_USERNAME}:$${GIT_PASSWORD}@github.com/BenSassiAhmed/gitops-pfe-2025.git"

        // Update manifest files with new image tags
        dir("gitops-pfe-2025/${MANIFESTS_DIR}") {
          sh """
            sed -i "s|image: ${SCRUM_APP_REPO}.*|image: ${SCRUM_APP_IMAGE}|g" scrum-app-deployment.yaml
            sed -i "s|image: ${SCRUM_UI_REPO}.*|image: ${SCRUM_UI_IMAGE}|g" scrum-ui-deployment.yaml
          """
        }

        // Commit and push changes
        dir("gitops-pfe-2025") {
          sh """
            git config --global user.name "Jenkins"
            git config --global user.email "jenkins@example.com"
            git add "${MANIFESTS_DIR}/scrum-app-deployment.yaml" "${MANIFESTS_DIR}/scrum-ui-deployment.yaml"
            git commit -m "Update image tags to ${BUILD_NUMBER}"
            git push https://$${GIT_USERNAME}:$${GIT_PASSWORD}@github.com/BenSassiAhmed/gitops-pfe-2025.git HEAD:main
          """
        }
      }
    }
  }
}

```

Figure 7.2: Updating Docker Version in GitOps Repo

We configured the pipeline to send an email notification at the end of each build. This notification included the build status (such as success or failure), along with a link to the Jenkins console output for easy debugging. This helped us stay updated and fix problems quickly if something went wrong.

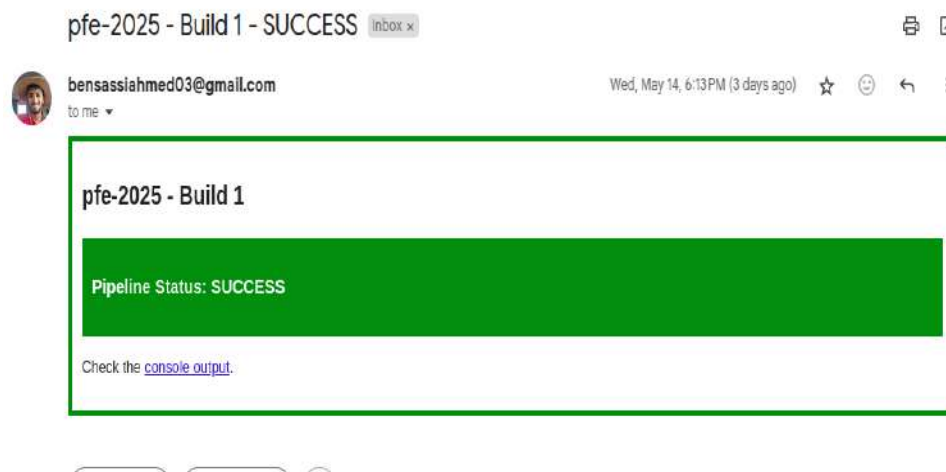


Figure 7.3: Email Notification

In ArgoCD, we set up a private GitOps repository to manage our Kubernetes manifests. First, we created a new private GitHub repository and pushed our application and infrastructure YAML files. Then we configured ArgoCD to let it access our private repository which we created.

CONNECT REPO USING HTTP/HTTPS

Type
git

Name (optional)
gitops-pfe-2025

Project
default

Repository URL
https://github.com/BenSassiAhmed/gitops-pfe-2025.git

Username (optional)
BenSassiAhmed

Password (optional)
.....

CONNECT SAVE AS CREDENTIALS TEMPLATE CANCEL

Figure 7.4: Setting GitOps Repository

The private GitOps repository was successfully set up and connected.

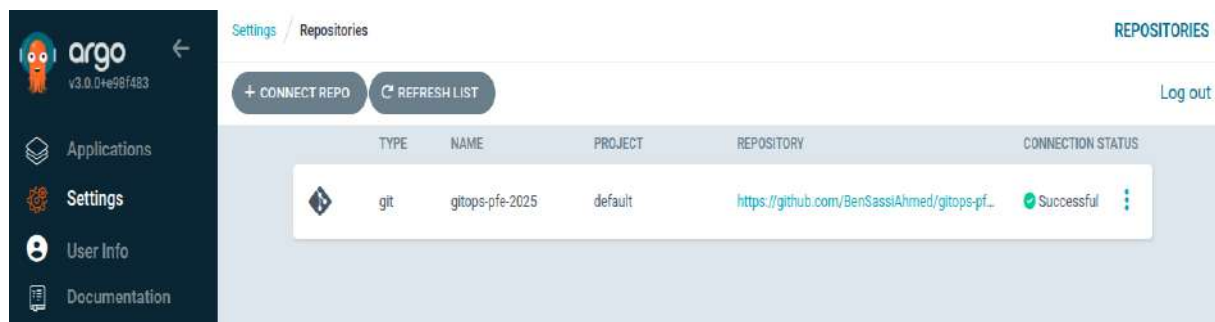


Figure 7.5: GitOps Repository Successfully Setup

After setting up the private GitOps repository in ArgoCD, we created a new application pointing to it. This enabled automated deployment and synchronization of our Kubernetes manifests directly from the repository.

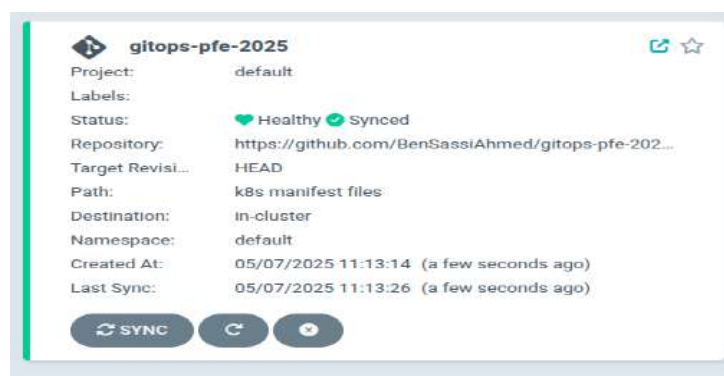


Figure 7.6: ArgoCD Application Created from Private GitOps Repository

The current running components are now visible through the ArgoCD interface. We can see all of our K8s components of our project!

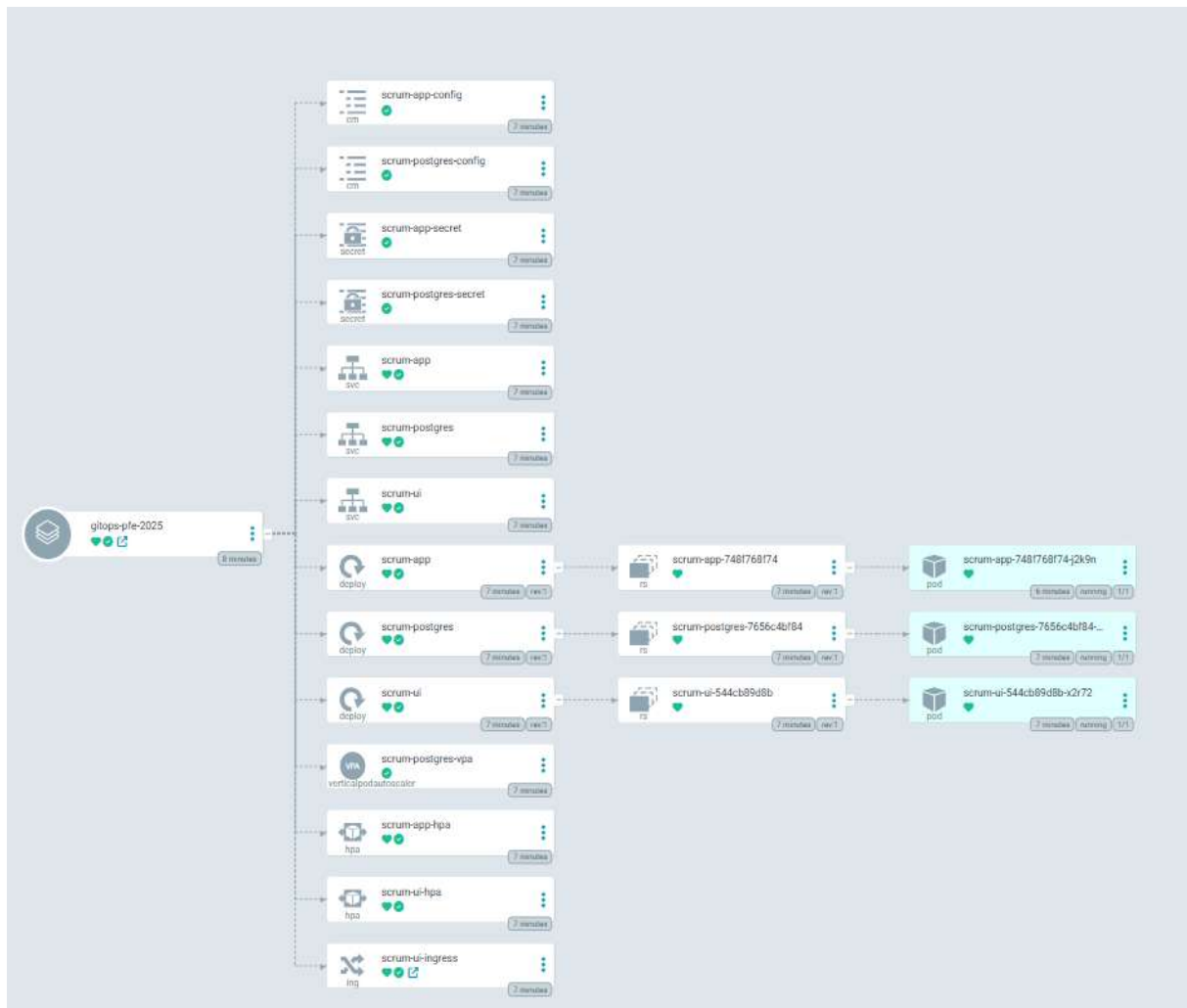


Figure 7.7: K8s components displayed in ArgoCD UI

In AWS console, we had initially the 3 nodes, 1 master and 2 worker nodes.

Instances (3) Info							
Last updated 2 minutes ago				Connect	Instance state	Actions	Launch instances
Find Instance by attribute or tag (case-sensitive)				All states			
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	worker-default-2025-0...	I-01f7d96b34d0cff1b	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3a
☐	control-plane	I-02eb1bfa30ee8928	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3a
☐	worker-default-2025-0...	I-0d7f4bf91d419e596	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3b

Figure 7.8: Initial Nodes in AWS Console

We verified the nodes in the terminal in our bastion node.

```
ubuntu@ip-172-32-10-233:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-172-32-10-233	Ready	control-plane	21m	v1.29.0
ip-172-32-110-132	Ready	<none>	19m	v1.29.0
ip-172-32-120-240	Ready	<none>	19m	v1.29.0

Figure 7.9: Current Kubernetes Cluster Nodes

Initially we had 3 pods , one for frontend one for backend and other one for the database

```
ubuntu@ip-172-32-10-233:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
scrum-app-5bf5b6bc7d-8ldqt	1/1	Running	0	7m16s
scrum-postgres-7656c4bf84-n99gk	1/1	Running	0	9m1s
scrum-ui-7ff54b8c5-pdvn2	1/1	Running	0	9m1s

Figure 7.10: Viewing Current Pods

The HPA setup of the Scrum UI and Scrum App is shown in this figure. We configured both applications to 1 replica. The target was 10 replicas. By having this setup, we could scale the applications based on the need.

```
ubuntu@ip-172-32-10-233:~$ kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
scrum-app-hpa	Deployment/scrum-app	0%/50%	1	10	1	11m
scrum-ui-hpa	Deployment/scrum-ui	1%/50%	1	10	1	11m

Figure 7.11: Initial HPA Configuration for Scrum UI and Scrum App

In this step, we confirmed the installation of essential components for the autoscaling features. We checked the kube-system namespace with the command `kubectl get pod -n kube-system` to confirm successful installation of AWS Load Balancer Controller, VPA, CA, and metrics server to enable the HPA. The image shows that they are successfully installed and ready.

```
ubuntu@ip-172-32-10-233:~$ kubectl get pod -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
aws-load-balancer-controller-f545fbb4f-6nd4m	1/1	Running	0	28m
aws-load-balancer-controller-f545fbb4f-vcxh2	1/1	Running	0	28m

Figure 7.12: Verification of AWS Load Balancer Controller

metrics-server-bccf467f-rbwzv	1/1	Running	0	30m
my-release-aws-cluster-autoscaler-b75c4cd8b-cg886	1/1	Running	0	40s
vpa-admission-controller-5f49bc9d4c-v7gf9	1/1	Running	0	28m
vpa-recommender-6fd9978498-b8565	1/1	Running	0	28m
vpa-updater-db6549ddf-zb8k5	1/1	Running	0	28m

Figure 7.13: Verification of Metric Server, CA and VPA

We went to one of the pods and created an infinite loop using the while command to stress-test the system to test the autoscaling features.

```
ubuntu@ip-172-32-10-233:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
scrum-app-5bf5b6bc7d-8ldqt          1/1     Running   0           22m
scrum-postgres-7656c4bf84-n99gk     1/1     Running   0           24m
scrum-ui-7ff54b8c5-pdnv2            1/1     Running   0           24m
ubuntu@ip-172-32-10-233:~$ kubectl exec -it scrum-app-5bf5b6bc7d-8ldqt -- /bin/sh
/ # while true; do ;; done
```

Figure 7.14: Stress Test on Pod to Trigger Autoscaling

Here as we can see, the number of pods are increased to handle more traffic. Once we have pods that are in a pending state, the CA will be triggered and the ASG will add additional worker nodes to our K8s cluster.

```
NAME                                READY   STATUS    RESTARTS   AGE
scrum-app-748f768f74-2zq74          1/1     Running   0           2s
scrum-app-748f768f74-8ktr7          1/1     Running   0           62s
scrum-app-748f768f74-9d5fs          0/1     Pending   0           2s
scrum-app-748f768f74-ccrwz          0/1     Pending   0           2s
scrum-app-748f768f74-ddpmm          0/1     Pending   0           2s
scrum-app-748f768f74-l4ssm          1/1     Running   0           62s
scrum-app-748f768f74-rpkch          1/1     Running   0           47s
scrum-app-748f768f74-s9z2t          1/1     Running   0           42m
scrum-postgres-7656c4bf84-25dzj     1/1     Running   0           43m
scrum-ui-544cb89d8b-ft26k           1/1     Running   0           43m
```

Figure 7.15: Pending State Pods

To handle the traffic efficiently, new nodes are been added automatically thanks to the autoscaling features of K8s.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
worker-default-2025-0...	i-01f7d96b34d0cff1b	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3a
control-plane	i-02eb1bfba30ee8928	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3a
worker-default-2025-0...	i-0aac9e81805d6c773	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3a
worker-default-2025-0...	i-0d7f4bf91d419e596	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3b
worker-default-2025-0...	i-0bf7a3e791d4c62bc	Running	t2.medium	2/2 checks passed	View alarms +	eu-west-3b

Figure 7.16: Increased Worker Nodes


```
ubuntu@ip-172-32-10-233:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-172-32-10-233                    Ready     control-plane  41m     v1.29.0
ip-172-32-110-132                   Ready     <none>      39m     v1.29.0
ip-172-32-110-175                   Ready     <none>      2m42s   v1.29.0
ip-172-32-120-214                   Ready     <none>      114s    v1.29.0
ip-172-32-120-240                   Ready     <none>      39m     v1.29.0
```

Figure 7.17: Nodes Terminal Verification

Now all the pods are in running state since we have added additional 2 worker nodes automatically without any manual intervention.

```
ubuntu@ip-172-32-10-233:~$ kubectl get pod
NAME                                READY    STATUS    RESTARTS   AGE
scrum-app-5bf5b6bc7d-7cfpk          1/1      Running   0           2m54s
scrum-app-5bf5b6bc7d-8ldqt          1/1      Running   0           26m
scrum-app-5bf5b6bc7d-ch72k          1/1      Running   0           2m54s
scrum-app-5bf5b6bc7d-gbwzq          1/1      Running   0           3m39s
scrum-app-5bf5b6bc7d-gq5xc          1/1      Running   0           3m9s
scrum-app-5bf5b6bc7d-k5gnr          1/1      Running   0           2m54s
scrum-app-5bf5b6bc7d-ns9xm          1/1      Running   0           2m9s
scrum-app-5bf5b6bc7d-pt4zj          1/1      Running   0           2m9s
scrum-app-5bf5b6bc7d-rmrfx          1/1      Running   0           3m39s
scrum-app-5bf5b6bc7d-xvw9k          1/1      Running   0           2m54s
scrum-postgres-7656c4bf84-n99gk     1/1      Running   0           28m
scrum-ui-7ff54b8c5-pdnv2            1/1      Running   0           28m
```

Figure 7.18: New Pods Created Automatically

We removed the traffic load, and after a short time, the extra pods were automatically deleted. Around 10 minutes later, the Cluster Autoscaler noticed that the two additional nodes it had previously added were no longer needed, so it removed them to optimize resource usage.

```
ubuntu@ip-172-32-10-233:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-172-32-10-233                    Ready     control-plane  62m     v1.29.0
ip-172-32-110-132                   Ready     <none>      61m     v1.29.0
ip-172-32-120-240                   Ready     <none>      61m     v1.29.0
```

Figure 7.19: Back With Initial Nodes State

Instances (5) Info							
		Last updated less than a minute ago		Connect	Instance state	Actions	Launch instances
Find Instance by attribute or tag (case-sensitive)		All states		< 1 > Settings			
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	worker-default-2025-0...	i-01f7d96b34d0cff1b	Running	t2.medium	2/2 checks passed	View alarms	eu-west-3a
☐	control-plane	i-02eb1bfba30ec8928	Running	t2.medium	2/2 checks passed	View alarms	eu-west-3a
☐	worker-default-2025-0...	i-0aac9e81805d6c773	Terminated	t2.medium	-	View alarms	eu-west-3a
☐	worker-default-2025-0...	i-0d7f4bf91d419e596	Running	t2.medium	2/2 checks passed	View alarms	eu-west-3b
☐	worker-default-2025-0...	i-0bf7a3e791d4c62bc	Terminated	t2.medium	-	View alarms	eu-west-3b

Figure 7.20: Scaling Down Nodes

We generated stress in the cluster, which caused a temporary spike in CPU and memory usage. Autoscaling was triggered to handle the load, and once the stress ended, resource usage returned to normal. The system stabilized as expected. Here is the dashboard of Grafana which visualize the metrics collected from Prometheus. Prometheus automatically collects data related to CPU and memory utilization, it is a built in solution.



Figure 7.21: Cluster Overview Dashboard

This graph shows CPU usage on one of the worker nodes. We saw a sharp peak around 22:42, which triggered the creation of new worker nodes through the cluster autoscaler. After that, usage dropped and the node returned to normal levels.

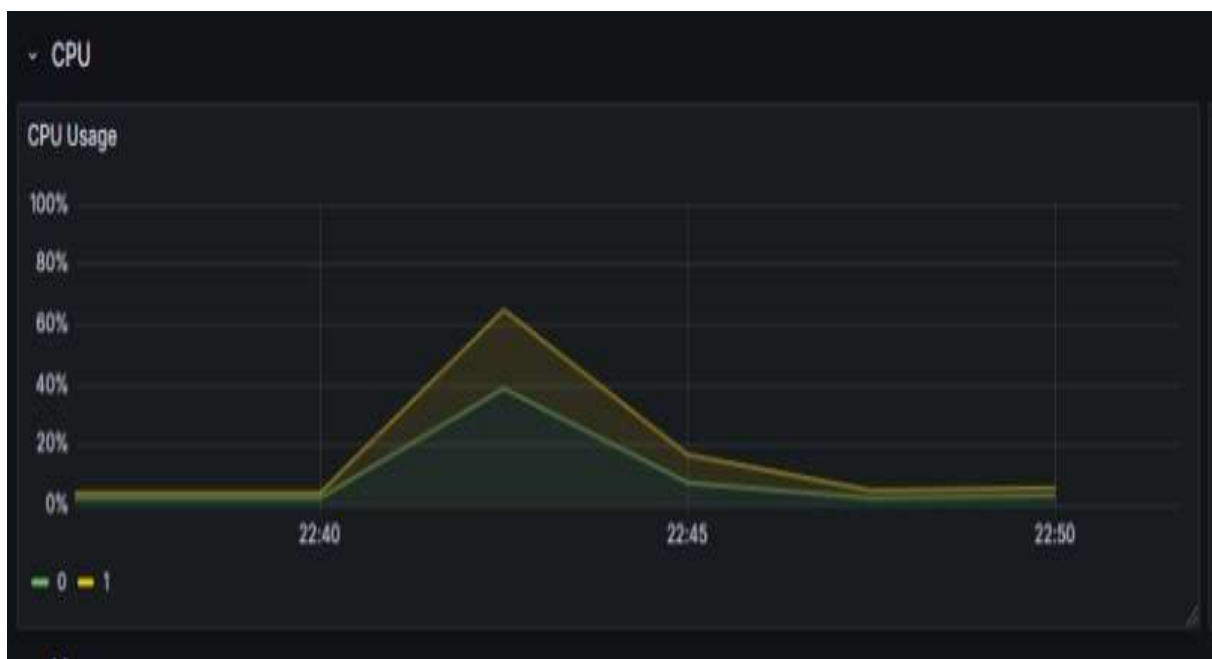


Figure 7.22: Worker Node CPU Usage

Conclusion

In this final sprint, we completed the continuous deployment phase by implementing GitOps using ArgoCD. This allowed us to automate and control deployments directly from the Git repository. We also tested the autoscaling features of our Kubernetes cluster, ensuring the system can adapt to varying loads. Finally, we set up real-time monitoring to observe the application's behavior and performance. These steps helped us build a reliable, scalable, and observable infrastructure, ready for production use.

General Conclusion and Perspectives

My final year project was an important step towards becoming a DevOps engineer. We had the chance to use the modern software engineering practices focused on automation, cloud-native design and continuous delivery.

The main aim of this work was to design and implement a complete DevOps pipeline which is automated end-to-end using IaC, CI, Containerization, Orchestration with Kubernetes, GitOps for deploying Application and Observability with Prometheus and Grafana. Everything was built on AWS ; This approach enabled us to utilise everything from the scalability, flexibility, and cost-efficiency.

In addition to the technology, we learned that planning, communication, and iterative delivery are essential. We learned how to break a big project down into manageable sprints along with organizing our work in a product backlog. We also learnt about Agile methodologies.

Through this hands-on experience, we gained an overview of popular DevOps tools. To benefit from orchestration, the uses of infrastructure-as-code, and the agility of CI/CD and GitOps workflows first-hand.

My project is mainly based on DevOps. We use it to automate the process of building and deploying an application. But we also recognize that in today's software development, security is very important. We would like to take it a step further by using DevSecOps in future. This means that security needs to be added at every step of the development process. The systems that we create should be fast, efficient, safe, and also secure. By learning and implementing DevSecOps, we can follow mainstream company best practices.

In summary, this project was both a technical and personal achievement. It gave us practical experience in building real DevOps solutions and the confidence to contribute to production-ready environments in the future. It was a strong foundation for our career as future DevOps and Cloud professional engineer.

Webography

- [1] <https://aws.amazon.com/devops/what-is-devops/> (Accessed on 02/20/2025)
- [2] <https://www.atlassian.com/devops/devops-tools> (Accessed on 02/20/2025)
- [3] <https://www.geeksforgeeks.org/what-is-ci-cd/> (Accessed on 02/20/2025)
- [4] <https://www.redhat.com/en/topics/containers/what-is-container-orchestration> (Accessed on 02/23/2025)
- [5] <https://www.codefresh.io/learn/gitops/> (Accessed on 02/23/2025)
- [6] <https://www.redhat.com/en/topics/cloud-computing/iaas-vs-paas-vs-saas> (Accessed on 02/26/2025)
- [7] <https://spacelift.io/blog/infrastructure-as-code> (Accessed on 02/26/2025)
- [8] <https://www.gyansetu.in/blog/what-is-docker-in-devops-how-does-it-work/> (Accessed on 03/04/2025)
- [9] <https://kubernetes.io/docs/concepts/overview/> (Accessed on 03/04/2025)
- [10] <https://argo-cd.readthedocs.io/> (Accessed on 03/05/2025)
- [11] <https://aws.amazon.com/what-is-aws/> (Accessed on 03/07/2025)
- [12] <https://developer.hashicorp.com/terraform/intro> (Accessed on 03/07/2025)
- [13] <https://www.devopsschool.com/blog/what-is-prometheus-and-how-it-works/> (Accessed on 03/10/2025)
- [14] <https://www.redhat.com/topics/data-services/what-is-grafana> (Accessed on 03/10/2025)

Abstract

This project is about building a DevOps solution that helps deploy and manage applications in the cloud using modern tools and methods. The project was carried out as part of the final year of the Telecommunications Bachelor's degree and follows an Agile approach structured around multiple development sprints. It includes the design and deployment of an automated CI/CD pipeline deployed on AWS, leveraging Infrastructure as Code (IaC) for provisioning the AWS infrastructure, containerization for application packaging, Kubernetes for orchestration and autoscaling, and GitOps for deployment. Observability was integrated to enable real-time monitoring. The primary goal was to build a scalable, automated, and efficient solution that supports continuous delivery from code to production.

Keywords: CI/CD, Docker, Kubernetes, Terraform, GitOps, ArgoCD, Prometheus, Grafana, Auto-scaling, AWS.

Résumé

Ce projet consiste à construire une solution DevOps permettant de déployer et gérer des applications dans le cloud en utilisant des outils et méthodes modernes. Le projet a été réalisé dans le cadre de la dernière année de la licence en Télécommunications et suit une approche Agile structurée autour de plusieurs sprints de développement. Il inclut la réalisation d'une pipeline CI/CD automatisée, utilisant l'Infrastructure as Code (IaC) pour la gestion de l'infrastructure AWS, la containerisation pour l'empaquetage de l'application, Kubernetes pour l'orchestration et l'autoscaling, ainsi que GitOps pour le déploiement. Une solution d'observabilité a été intégrée pour permettre la surveillance en temps réel. L'objectif principal était de construire une solution évolutive, automatisée et efficace, supportant la livraison continue du code jusqu'en production.

Mots-clés : CI/CD, Docker, Kubernetes, Terraform, GitOps, ArgoCD, Prometheus, Grafana, Auto-scaling, AWS.